

2022-09-12

Automated Bug Severity Prediction using Source Code Metrics, Static Analysis, and Code Representation

Mashhadi, Ehsan

Mashhadi, E. (2022). Automated Bug Severity Prediction using Source Code Metrics, Static Analysis, and Code Representation (Master's thesis, University of Calgary, Calgary, Canada).

Retrieved from <https://prism.ucalgary.ca>.

<http://hdl.handle.net/1880/115221>

Downloaded from PRISM Repository, University of Calgary

UNIVERSITY OF CALGARY

Automated Bug Severity Prediction using Source Code Metrics, Static Analysis, and Code Representation

by

Ehsan Mashhadi

A THESIS

SUBMITTED TO THE FACULTY OF GRADUATE STUDIES
IN PARTIAL FULFILMENT OF THE REQUIREMENTS FOR THE
DEGREE OF MASTER OF SCIENCE

GRADUATE PROGRAM IN ELECTRICAL ENGINEERING

CALGARY, ALBERTA

SEPTEMBER, 2022

© Ehsan Mashhadi 2022

Abstract

In the past couple of decades, significant research efforts are devoted to the prediction of software bugs. However, most existing work in this domain treats all bugs the same, which is not the case in practice. It is important for a defect prediction method to estimate the severity of the identified bugs so that the higher severity ones get immediate attention. In this thesis, we provide a quantitative and qualitative study on two popular datasets (Defects4J and Bugs.jar), using 10 common source code metrics, and also two popular static analysis tools (SpotBugs and Infer) for analyzing their capability in predicting defects and their severity. We studied 3,358 buggy methods with different severity labels from 19 Java open-source projects. Results show that although code metrics are powerful in predicting buggy code, they cannot estimate the severity level of the bugs. In addition, we observed that static analysis tools have weak performance in both predicting bugs (F1 score range of 3.1%-7.1%) and their severity label (F1 score under 2%). We also manually studied the characteristics of the severe bugs to identify possible reasons behind the weak performance of code metrics and static analysis tools. Also, our categorization shows that Security bugs have high severity in most cases while Edge/Boundary faults have low severity. Furthermore, we show that code metrics and static analysis methods can be complementary in terms of estimating bug severity. For finding the effectiveness of machine learning models in predicting bug severity, we train 8 different models on code metrics only as a baseline and evaluate them based on different evaluation metrics. The overall result was not promising, but the Decision Tree and Random Forest models have better results. Then, we leveraged the pre-trained CodeBERT model to use code representation by feeding the source code input only, and the results improved significantly in the range of 29%-140% for different metrics. We also integrated code metrics into the CodeBERT model by providing two architectures named ConcatInline and ConcatCLS which enhance the CodeBERT model efficacy.

Contributions

During my study, I have made contributions in the form of research papers, datasets, and providing the replication package of my studies. I would like to quickly enumerate them here:

1. Mashhadi, Ehsan, and Hadi Hemmati. "Applying codebert for automated program repair of java simple bugs." In 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR), pp. 505-509. IEEE, 2021. [79]
2. Mashhadi, Ehsan, Shaiful Chowdhury, Somayeh Modaberi, Hadi Hemmati, and Gias Uddin. "An Empirical Study on Bug Severity Estimation Using Source Code Metrics and Static Analysis." arXiv preprint arXiv:2206.12927 (2022) [78]. (Under Review)
3. Providing the bug severity dataset containing real bugs of popular Defects4J and Bugs.jar datasets <https://github.com/EhsanMashhadi/EMSE-BugSeverity>

Acknowledgements

I want to appreciate the support from my supervisor, Dr. Hadi Hemmati for all his helpful advice during my studies.

I want to thank my wife who accompanies me during my studies and immigration process.

I also want to thank my family and my friends who support and encourage me during these years.

Table of Contents

Abstract	ii
Contributions	iii
Acknowledgements	iv
Table of Contents	vi
List of Figures	vii
List of Tables	viii
List of Symbols, Abbreviations, and Nomenclature	ix
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.3 Objectives and RQs	3
1.4 Contributions and Thesis Organization	4
2 Background and Related Work	5
2.1 Bug Taxonomy	5
2.2 Bug Severity	6
2.3 Bug Prediction	6
2.4 Bug Severity Prediction	7
2.5 Static Analysis Tools	8
2.6 Source Code Metrics	8
2.7 Evaluation Metrics	11
2.8 Qualitative Study	13
2.8.1 Grounded Theory	14
2.8.2 Card Sorting	14
2.9 Representation Learning	14
2.9.1 Embedding	14
2.9.2 Word Embedding	15
2.9.3 Code Embedding	16
3 Proposed Approach	18
3.1 The Study Architecture	18
3.2 Dataset Collection	20

4	Empirical Evaluation	23
4.1	Data Collection	23
4.1.1	Defects4J	23
4.1.2	Bugs.jar	24
4.1.3	Buggy vs. Not Buggy	25
4.2	Experiment Design and Results	27
4.2.1	Code Metrics & Bug Severity (RQ1)	27
4.2.2	Static Analysis Tools & Bug Severity (RQ2)	34
4.2.3	Manual Analysis of Bug Characteristics (RQ3)	39
4.2.4	Representation Learning & Bug Severity (RQ4)	53
4.3	Discussion	65
4.4	Threats to Validity	68
4.4.1	External Validity	68
4.4.2	Internal Validity	69
4.4.3	Construct Validity	69
4.4.4	Conclusion Validity	69
5	Conclusion and Future Work	71
	Bibliography	73

List of Figures

2.1	Description of ROC and Precision-Recall curve plots in different cases.	13
3.1	The overall approach for our study on bug severity prediction.	19
3.2	Dataset collection approach and required changes for each RQ.	22
4.1	Buggy methods severity distributions of Defects4J and Bugs.jar datasets with their USL values.	25
4.2	Buggy methods severity distributions in Defects4J dataset with the USL values.	26
4.3	Buggy methods severity distributions in Bugs.jar dataset with the USL values.	26
4.4	Comparing source code metrics between buggy methods (b axis) and non-buggy methods (nb axis) using aggregated dataset.	30
4.5	Comparing source code metrics between different severity groups for Defects4J and Bugs.jar datasets.	33
4.6	Confusion matrices of bug severity prediction by SpotBugs for Defects4J and Bugs.jar datasets. N/A means that the tool does not predict a value. For example, in the Defects4J dataset, 23 critical bugs have not been detected. Colors show the magnitude of numbers.	38
4.7	Sample bug with High severity value in the JXPath-14 project of the Defects4J dataset.	45
4.8	Sample bug with High severity value in the Math-36 project of the Defects4J dataset.	45
4.9	Sample bug with the Major severity value in the OAK-3324_5f863af6 project of Bugs.jar dataset.	46
4.10	Sample bug with the Low severity value in the Closure-119 project of the Defects4J dataset.	47
4.11	Sample bug with the Low severity value in the Lang-30 project of the Defects4J dataset.	48
4.12	Critical bug in the Cli-2 project of the Defects4J dataset.	50
4.13	Critical bug in the Math-11 project of the Defects4J dataset.	51
4.14	ROC curve of baseline models.	58
4.15	Precision-Recall curve of baseline models.	60
4.16	Confusion matrix of baseline models.	62
4.17	ROC curve and Precision-Recall curve of CodeBERT model.	62
4.18	Confusion matrix of CodeBERT model.	63
4.19	Confusion matrix of ConcatInline and ConcatCLS models.	64
4.20	ROC curve and Precision-Recall curve of ConcatInline and ConcatCLS models.	66

List of Tables

2.1	List of studied metrics and their brief description.	11
4.1	Unifying the raw severity labels of Defects4J dataset. For example, the third row means that bugs having raw severity labels of Medium or 5 are considered as Medium bugs.	24
4.2	Merging different USLs to create categories for Defects4J and Bugs.jar datasets. For example, the first row means that bugs having Critical or High USLs are considered in the Critical category.	28
4.3	Cliff's Delta Effect sizes for comparing code metrics between buggy and non-buggy methods. S refers to Small and M refers to Medium effect size.	29
4.4	Wilcoxon Rank-sum test results to show the percentage of projects where code metrics exhibit statistically difference between buggy and non-buggy methods.	31
4.5	Cliff's Delta Effect sizes after comparing buggy vs non-buggy methods. N refers to Negligible, S refers to Small, M refers to Medium, and L refers to Large effect size. For example, the first column of the first row compares the LC distributions between buggy and non-buggy methods and shows the percent of differences with negligible, small, medium, and large effect sizes.	31
4.6	Cliff's Delta Size for bugs with different severity values. α was set to 0.05. A red cell indicates that the null hypothesis is rejected (i.e., the distributions are statistically significantly different), whereas a blue cell means they are not statistically different. Effect sizes are N~Negligible, and S~Small. For example, the first cell shows that in the Defects4J dataset, the distribution of source lines of code (LC) between methods with high and low severe bugs is not statistically different (blue) and the difference has a negligible effect size.	34
4.7	Translation between the SpotBugs reported rank (SRR) value and the unified severity label (USL) value.	36
4.8	Static analysis tools performance in finding buggy methods.	37
4.9	SpotBugs performance in finding bug severity of Defects4J and Bugs.jar datasets.	39
4.10	Two sample bugs from Severe and Non-Severe categories with their code metric values, normalized values, and the Sum Complexity values. The value in each row and the value of Sum Complexity are rounded to two precision.	41
4.11	Distribution of randomly sampled bugs with low-level bug labels.	43
4.12	Distribution of high-level bug labels where there is contradiction between severity values and Sum Complexity (SC) value.	44
4.13	Mapping USL types to the numerical format for being used as a target variable in the training process.	54
4.14	Baseline models performance with different evaluation metrics. The bold line shows the model with the best performance.	58
4.15	CodeBERT model performance with different evaluation metrics.	58
4.16	Creating context for code metric features to provide as the NL part of the model.	63
4.17	ConcatInline and ConcatCLS models performance with different evaluation metrics.	65

List of Symbols, Abbreviations, and Nomenclature

Symbol	Definition
D	Difficulty metric
E	Effort metric
R	Readability metric
FO	FanOut metric
LC	Lines of Code metric
MA	McCabe metric
MI	Maintainability Index metric
ML	McClure metric
PI	Proxy Indentation metric
SC	Sum Complexity
AUC	Area Under Curve
D4J	Defects4J dataset
FPR	False Positive Rate
MCC	Matthews Correlation Coefficient
NBD	Nested Block Depth metric
ROC	Receiver Operating Characteristic
RSL	Raw Severity Label
SRR	SpotBugs Reported Rank
TPR	True Positive Rate
USL	Unified Severity Label

Chapter 1

Introduction

1.1 Motivation

Software maintenance is one of the most challenging and expensive parts in the software development life cycle [60]. Handling bugs (including detecting, localizing, fixing, etc.) is the most typical challenge associated with the software maintenance step [25, 23]. Consequently, both practitioners and researchers are trying to make this tedious task as automated as possible from different aspects such as defect prediction, test generation, fault-localization, and program repair [65, 113, 124, 140, 79].

While there has been much research in handling bugs using different techniques like search-based [70], pattern-based [73] and ML-based techniques [79], there is little research that focuses explicitly on the severe bugs. In other words, most of the research implicitly assumes that all bugs have the same importance [110, 79, 135, 98]. However, the bug severity indicates the intensity of the impact the bug has on system operation [89]. Critical bugs may cause a system to crash completely or cause non-recoverable conditions such as data loss. High-severity bugs affect major system components that prevent users from working with some parts of the system. Fixing severe bugs are typically more challenging compared to the medium or low severity bugs, where few components are affected and there is an easy workaround [128]. Therefore, in practice, bugs with higher severity should be fixed sooner than other less severe bugs to reduce the potential harm [126, 61, 106].

In general, software practitioners have several means to detect bugs. These can range from QA practices such as code review [66, 75, 19], and inspection to different testing approaches, and even development methodologies such as pair programming [134, 88, 119], and Test Driven Development (TDD) [24, 76, 15, 102]. When a bug is detected, it is common for the development teams to consider the most severe/important bugs

first which helps them to prevent extreme consequences. Some issue tracking systems, such as Jira¹, have a specific field named Severity/Priority which is assigned during the bug reporting, and helps the development teams to consider bug importance during debugging.

It is important to note that bug severity does not have a standard formal definition. The definition depends on the context of the software, the nature of the bug, the current state of the project, the ratio of affected users, potential harm to users, and many other factors. The current state of the practice to identify and record the severity of bugs is through a manual process in issue tracking systems, where the severity of a bug has its own field (with options such as *Blocker*, *Critical*, *Major*, *Minor*, and *Trivial* or sometimes with numbers ranging from zero to 20). The field is manually populated by the person who documents the bug, however, it may change by the technical team during the bug reporting review process. Thus, the whole process is not only quite subjective but is also an error-prone task.

1.2 Problem Statement

Therefore, it seems that an approach that can estimate the severity of a given bug, automatically and systematically (not biased to the opinion of the bug reporter), can be quite helpful not only for the developers and QA teams but also as an input for other automated software tools, such as program repair, to prioritize their process, accordingly. However, the current literature in this domain is limited to predicting the severity based on the bug report description using NLP techniques [29, 121], which still has the bias issue (subjective to reporter’s opinion) and ignores the context (underlying source code). Also, their applicability is limited to projects with rich and informative bug reports. Therefore, in this study, we would like to empirically investigate the potential of source code for estimating the severity of a bug.

To be able to estimate bug severity, we study three well-known categories of approaches that have a track record of predicting issues in the code: (a) source code metrics, (b) static analysis, and (c) machine learning models.

Code metrics have been used for various applications in the previous research and the provided results show that these metrics are good enough to predict code smell [125], maintenance effort [100] and defects [41, 39].

Static analysis tools such as SpotBugs [9], the successor to the popular FindBugs [4] tool, Facebook Infer [6], and Google Error Prone [3] have also been successfully used in related research [45, 123]. These tools use different techniques such as AST-based patterns or data-flow analysis to find a bug’s existence and to predict the bug’s type and severity. These tools are updated regularly to cover modern bug patterns and they

¹<https://www.atlassian.com/software/jira>

require the compiled code to perform the analysis which takes longer than the source code metric calculation (e.g, several hours compared to several minutes) which makes it less practical for big projects.

Machine learning models including classic models (such as SVM, Decision Tree, Random Forest) and deep learning models (such as MLP, LSTM, Transformers [129]) have been used to predict the bug severity [105]. In recent studies, pre-trained models such as CodeBERT [38] (a large language model based on Transformers) have been proposed that can be used on multiple tasks including bug detection.

However, to the best of our knowledge, there is no study that explores code metrics, modern static analysis tools, or code representation’s ability in estimating bug “severity”. Our approach focuses on method-level granularity in contrast to class/module-level, since researchers and developers find the class/module level granularity too coarse-grained for practical usage [112, 97, 44, 50].

1.3 Objectives and RQs

We use two popular datasets: Defects4J [59], and Bugs.jar [107] which contain real bugs from different open-source Java projects. We studied 19 projects containing 1,668 bugs (3,358 buggy methods) for our quantitative study. Furthermore, we studied 60 randomly sampled bugs from both datasets for our qualitative study to find the bug characteristic according to their severity labels.

To guide our study, we target answering the following research questions (RQ):

- **RQ1:** Are source code metrics good indicators of buggyness and bug severity?

Answer: The results show that most of the code metrics (e.g, Lines of Code, McCabe, McClue, Nested Block Depth, Proxy Indentation, FanOut, Readability, Difficulty, and Effort) are good indicators of buggyness, but they perform quite poorly for predicting bug severity. Line of Code has the best performance in finding buggyness, but the least performance in finding bug severity. However, the Difficulty and Effort metrics are the best indicators of bug severity.

- **RQ2:** What is the capability of static analysis tools in finding bugs and their severity?

Answer: The results show that the studied static analysis tools (SpotBugs and Infer) are not yet powerful enough to find many bug types and in many cases, they miss or mislabel the bug severity.

- **RQ3:** What are the characteristics of bugs with different severity values?

Answer: Results reveal that the severity of bugs is mostly related to the software’s specification, which is not predictable solely based on method-granularity level analysis. Also, we found no direct relationship between method complexity and its severity value. Many low severity bugs exist in the

quite complex methods according to the code metrics, but these functions handle trivial functionalities, such as GUI, or they do not lead to a crash or unauthorized access.

- **RQ4:** Can the code representation provided by the CodeBERT model predict the bug severity?

Answer: Our results show that the source code representation (embedding) provided by the CodeBERT model can predict the bug severity significantly better than the 8 baseline models using source code metrics as features. Also, we found that adding numerical code metric features to the CodeBERT model with two different provided architectures (ConcatInline and ConcatCLS) further improves the efficacy of the CodeBERT model. Our comparison shows that the ConcatCLS model outperforms the ConcatInline model since it has a better MCC value and its confusion matrix shows that the number of miss-classification in the case of predicting lower-severity labels is reduced.

1.4 Contributions and Thesis Organization

The findings of this study help researchers and practitioners to better understand the characteristics of severe bugs and guide future research/tools on how to advance the field to better predict bug severity.

The remainder of the thesis is organized as follows: we provide the background materials and related work in Chapter 2. We propose a general overview of our approach in Chapter 3. Dataset pre-processing, experiment motivation, design, results, threats to validity, and discussions about the results are discussed in Chapter 4. In Chapter 5 we conclude the study with possible future work.

Chapter 2

Background and Related Work

2.1 Bug Taxonomy

Finding the bug type is the very first and helpful step in the bug triaging phase since it will illuminate the technical team to handle the bug more easily and accurately. Although there is no common definition of bug types that can be applied to different projects, there has been much research that tries to categorize bugs based on their behavior and their impacts on the underlying systems [11].

The research provided by Chillarege et al. [30] is the earliest research for finding the bug taxonomy where the authors provide eight defect type such as Function (bugs in the code that implement a function), Interface (bugs in interacting with other components/modules), Checking (bugs related to cases where data and values are not validated properly before they are used), Assignment (errors indicating a few lines of code such as the control blocks initialization), Time/Serialization (bugs occur where shared and real-time resources are not used correctly), Build/Package/Merge (errors in build systems or version control), Documentation (bugs that affect publications/maintenance documentation), and Algorithm (bugs in the implementation of a problem that can be fixed without the need for change in the design of the system).

Catolino et al. [28] proposed an empirical assessment of the possible bug types by providing a novel taxonomy of bug types. They analyzed 1280 bug reports of 119 software projects of three large ecosystems including Apache, Mozilla, and Eclipse. They concluded with nine different bug types such as Configuration issue, Network issue, Database-related issue, GUI-related issue, Performance issue, Permission/deprecation issue, Security issue, Program anomaly issue, and Test code-related issue. They also found that these bug types are treated differently in different teams based on the procedure that the technical team follows to fix them.

2.2 Bug Severity

Bug severity is one of the bug features that are assigned manually by the QA/developers during the bug report or triaging stages. It is one of the most important factors that help teams to decide how fast the bugs should be fixed [136]. In other words, bugs that have high severity values (e.g, Critical/Blocker) should be fixed sooner than bugs having low severity values (Minor, Trivial). Different issue management systems leverage different labels for the bug severity field, and also technical team may add some custom labels themselves with custom meanings based on their requirements. In this section we provide the definition of some of the most popular severity values based on the [136, 1]:

Blocker: This category shows the most severe bugs that have the highest impacts on the systems and lead to irreparable damage to the system.

Critical: Bugs having this severity value can shut down the whole system, so this impact all users and may have very harmful consequences.

Major, High: These bugs can collapse most or very large parts of the systems so this will affect many users.

Minor, Low: These bugs can lead to unexpected/undesired behaviors that some users may notice, but this will not disrupt the system functionality. The impacts of these bugs are low since users can use the system even when these bugs are present.

Trivial: These are the bugs that will not lead to any noticeable breakdown or change in the systems, so their impacts are negligible.

Enhancement: This severity category usually does not show a bug, and they are used for issues that address enhancement to the functionality of part of the system. Since We do not have this severity type in our datasets we did not study this type in our experiments.

2.3 Bug Prediction

There has been much research in the bug-prediction field by focusing on different aspects using diverse granularity levels and various techniques [47, 55, 129, 71]. Previous studies can be categorized from different aspects such as working on the characteristics of source code [64, 94, 20, 21, 93, 116, 103] or the bug description [120, 130, 63]. From the source code granularity aspect, there has been research in file level [141, 42], package level [141], class level [114], and method level [41]. For bug severity prediction task, almost all of the existing works have leveraged bug report using natural language techniques or different classic and deep neural network models. In the rest of this section, we briefly discuss the related work on generic bug

prediction, bug severity prediction, and static analysis tools.

Bug prediction studies have used different code metrics such as LOC [97, 16, 113, 31], McCabe [16], Halstead metrics [17], C&K metrics [118, 58]. Code metrics have been applied on different granularity levels such as package/class level [91, 67, 141], method level [97, 41, 40, 31, 44, 87], and line level [133]. One major problem with the high granularity levels (package/class) is that they are practically less helpful for the developers [41, 97] since it requires significant effort to locate bugs at the package/class components. Unfortunately, line-level granularity can suffer from too many false positives, because multiple lines can be similar just by chance [117, 109]. Consequently, method level granularity has been the new focus to the community [97, 41, 40, 31, 44, 87], especially for developing bug prediction models, and several studies show positive and encouraging results [41, 40, 87].

2.4 Bug Severity Prediction

Several researchers leverage the bug report description as input and use different natural language processing techniques to predict the bug severity labels. Tian et al. [122] leveraged bug reports and their severity labels that are available in the history of projects to recommend severity labels for newly reported bugs. They measure the similarity between bug reports and use nearest neighbors classification. Their extensive experiments on thousands of bug reports taken from three various software systems show promising results by improving the f-measure of the state-of-the-art approach, especially on hard-to-predict severity labels by having up to 76.3% for the f-measure metric. Ramay et al [105] proposed an approach by using a deep learning model, natural language techniques, and emotion analysis by using bug reports for predicting bug severity. Their reported results show the f-measure up to 88.01% which means that this method improved the efficacy of the state-of-the-art approaches by increasing the f-measure metric by 7.90% on average. Lamkanfi et al [139] proposed the bug severity prediction by using the Topic Modeling technique. Their evaluation of 30,000 bug reports extracted from Eclipse, Mozilla, and Netbeans projects shows the effectiveness of their approach in predicting severity. They have reported the f-measure of 65% to 75% on Bugzilla reports. Tan et al [121] leveraged question-and-answer pairs available on the Stack Overflow website and combine them with related bug reports to make an enhanced version of the bug reports. They predict the severity labels of bug reports using different machine learning models which led to improvements by approximately 23% of the average f-measure metric.

2.5 Static Analysis Tools

Static analysis tools have been introduced for finding bugs by using different techniques, such as pattern matching. **FindBugs** [4] is an open-source tool that analyzes Java byte code for finding bugs. **SpotBugs** [9] is a successor of FindBugs which is actively under maintenance. SpotBugs has more than 400 bug patterns and their description is available on the tool’s website [9]. **Infer** [6] is a static analysis tool for different programming languages such as Java, C++, Objective-C, and C, which is also actively maintained. Infer has more than 100 predefined issue types which are described on its website [6]. **Error prone** [3] is another static analysis tool from Google which is used for Java code to catch common programming faults at the compile-time, and it is actively under maintenance. We have selected SpotBugs and Infer tools for our study because of their extensive usage in the previous studies and in practice, and we exclude the ErrorProne because its latest versions do not support older versions of Java projects which exist in our datasets. A more detailed explanation of our selected configuration for these tools is explained in Section 4.2.2.

Various research has been done to evaluate the true effectiveness of various static analysis tools for bug detection tasks. Some researchers leveraged these tools as an oracle for their own provided techniques. For example, Tomassi [123] used SpotBugs and ErrorProne to detect bugs in the BugSwarm dataset, and they found that these tools are not effective in finding the bugs since their results show only one successful bug detection by SpotBugs. Ayewah et al. [18] leveraged FindBugs for finding bugs in Google’s internal code-base, and they found that integrating this tool into Google’s Mondrian code review system would help developers see the potential bug existence in the code. Habib and Pradel [45] studied SpotBugs, Infer, and Google Error Prone tools to find their capabilities in detecting Java real bugs. They concluded that these tools are mostly complementary to each other, and they miss the large majority of the bugs. Dura et al. [36] introduced the JavaDL which is a Datalog-based declarative specification language for bug pattern detection in Java code, and compare it against the SpotBugs and ErrorProne tools. Authors found that JavaDL has comparable performance to these tools. Habib and Pradel [46] proposed a method to consider bug detection as a classification problem by using the neural networks and using Google Error Prone as an oracle. They show that their approach is effective for some bug patterns, sometimes reaching a precision and recall of over 80%, but also that it has not good efficacy in understanding some program properties.

2.6 Source Code Metrics

Different source code metrics with module-level, class-level, and method-level granularities have been used to measure software quality and predict bugs in previous research [41, 99]. These code metrics have different

advantages (e.g., being fast to calculate) and disadvantages (e.g., requiring a language-specific parser). Some of these metrics are introduced to cover the previous code metrics’ weaknesses. For example, McClure [83] which was proposed to improve the McCabe [82] by considering the number of control variables. In this study, we leverage most of the common method-level source code metrics that are used in the previous research (mentioned in the Related Work section) to see their capability in predicting buggy codes and their severity. Although the list of selected metrics is not exhaustive (given the numerous metrics explored in this field and the limitations of experiment size in one article), but we made sure we have most of the metrics that have been shown effective in predicting the method-level bugginess, in the past. These metrics are defined as follows:

Lines of Code (LC): Size, also known as lines of code (LC), is the most popular, easy to measure, and the most effective code metric for estimating software maintenance [43, 37, 31]. The use of LC as a proxy maintenance indicator is so prevalent that there are dedicated studies that completely focus on LC and its correlation with other quality metrics (e.g., [43, 68, 31]). LC has been extensively studied for bug prediction, fault localization, and for finding vulnerabilities (e.g., [97, 16, 113, 31]). In this study, we calculate LC as the source lines of code without comments and blank lines, similar to [68, 104, 31] to prevent the code formatting and comments effects, which are out of this study scope.

McCabe (MA): McCabe [82, 68], also known as cyclomatic complexity, is another very popular metric that indicates the number of independent paths, and thus the logical complexity of a program. Intuitively, components with high McCabe values are more bug-prone. McCabe has been studied extensively to find bugs and locate suspicious code [16], to understand its correlation with code quality [96], and to leverage its value for test generation methods, such as structured testing (path testing) [132]. McCabe can be calculated as $1 + \#predicates$ [82].

McClure (ML): McClure [83, 60] was proposed as an improvement over McCabe. Unlike McCabe, McClure considers the number of control variables, and the number of comparisons in a predicate, which is not supported by McCabe. Intuitively, a predicate with multiple comparisons and multiple control variables would be more complex, and thus more bug-prone, than a predicate with only one comparison or only a single control variable.

Nested Block Depth (NBD): McCabe and McClure do not consider nested depth. According to these metrics, there is no difference between two code snippets containing two identical for loops if they are arranged serially or nested. NBD [62, 12, 137] has been studied alongside McCabe and McClure to alleviate this issue.

Proxy Indentation (PI): Since McCabe-like complexity measures require a language-specific parser (for finding the predicates), Hindle et al. [54] proposed Proxy Indentation metric as a proxy for McCabe-like

complexity metrics. It was shown that, for measuring complexity, indentation measurement can perform very similar to more complex measurements such as McCabe, without requiring a language-specific parser. Indentation measurement is done for each line, and then an aggregated value is calculated for the whole program component (e.g., a method). Hindle et al. showed that the standard deviation as an aggregated value outperforms mean, median or max.

FanOut (FO): This metric calculates the total number of methods called a given method. This provides an estimate of the coupling—i.e., dependency of a particular method on other methods. It is observed that code components that are highly coupled are less maintainable and bug-prone [86, 13].

Readability (R): This metric combines different code features to calculate a single value for estimating code readability. We used the readability metric proposed by Buse et al. [27] which generates a readability score for a given method. The readability scores range from 0 to 1 for specifying least readable code to most readable code, respectively. The authors concluded that this metric has a significant level of correlation with defects, code churn, and self-reported stability.

Halstead Metrics: The Halstead code metrics contain seven measures based on the number of operators and operands in a component [48]. These metrics have been used in different research such as measuring code complexity perceived by developers [17], calculating the complexity of software maintenance tasks [33, 60], finding their correlation with indentation measures [54], and estimating software readability [101]. As all the Halstead metrics are highly correlated to each other, so in this study, we consider only two of them: **Difficulty (D)** and **Effort (E)** which use other Halstead metrics in their formulas. The Halstead Difficulty is calculated as:

$$D = \frac{n1}{2} * \frac{N2}{n2}$$

Where $n1$ is the number of distinct operators, $n2$ is the number of distinct operands, and $N2$ is the total number of operands.

The Halstead Effort is calculated as:

$$E = D * V$$

$$V(HalsteadVolume) = N * \log_2(n)$$

$$N = N1 + N2$$

$$n = n1 + n2$$

Where $N1$ is the total number of operators.

Maintainability Index (MI): Maintainability Index has been introduced by Omran and Hagemeister [92] where the authors defined metrics for measuring the maintainability of a software system and combine those metrics into a single value. This metric has evolved over time and was adopted by popular tools like Visual Studio [10]. MI can be calculated as:

$$171 - 5.2 \times \ln(HalsteadVolume) - 0.23 \times (McCabe) - 16.2 \times \ln(LC)$$

Where *HalsteadVolume* and *LC* are defined previously in this section.

Table 2.1 provides a brief description for each of our 10 studied metrics with their abbreviation.

Table 2.1: List of studied metrics and their brief description.

Metric	Description
LC	Number of source code lines without comments and blank lines
MA	Number of independent paths (logical complexity)
ML	Number of control variables and number of comparisons in a predicate
NBD	Counting the depth of most nested block
PI	Counting indentation of source code lines
FO	Counting total number of methods called a given method
R	Measuring the readability of the code in the range of 0-1 (least to most)
D	Measure the difficulty of the program to write or understand the code
E	Determining the required effort for developing/maintaining the code
MI	Measuring how much the code is maintainable

2.7 Evaluation Metrics

We use different evaluation metrics for assessing our trained models. Since our dataset is imbalanced, like many real datasets, we cannot rely on the accuracy metric and we provided several metrics to make our comparison robust.

Accuracy: Accuracy is a popular metric that exhibits the proportion of correct predictions among the total number of examined cases. It is a helpful metric for the balanced dataset, but in the case of imbalanced dataset other metrics such as F1 or MCC which are explained later are more valuable.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

F-measure: F-measure or F-score is a popular evaluation metric in classification tasks including software engineering tasks such as bug detection and severity detection. It considers both precision and recall, and specifically, F1 is the most common form of F-measure, which takes the harmonic mean of precision and recall.

$$Precision = \frac{TP}{TP + FP}$$

$$Recall(TPR) = \frac{TP}{TP + FN}$$

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

F1-Weighted: For multi-class problems, we cannot use the F1 score directly, but there are different options for handling this issue. In our case where our dataset is imbalanced, we use the “Weighted” format which is calculated by taking the mean of all per-class F1 scores and also considering each class sample number. In this way, the output value has accounted for each class contribution by considering the number of instances in that given class.

ROC Curve Plot: The Receiver Operator Characteristic (ROC) is a curve that plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold values.

$$FPR = \frac{FP}{FP + TN}$$

As it is shown in Fig 2.1, a perfect model is represented by a line from the bottom left to the top left and then across the top right of the plot. A skillful model is represented by a curve that bows up to the top left of the plot. However, a random classifier (no-skill) is represented by a diagonal line from the bottom left to the top right of the plot.

AUC-ROC: The Area Under the Curve (AUC) is the measure of the ability of a classifier to distinguish between classes by indicating how well the probabilities from the positive classes and the negative classes are separated. The perfect classifier has an AUC score of 1 which means it distinguishes between the positive and negative classes well while the classifier with an AUC score of 0.5 means it works like a random classifier and it has no ability in finding the positive and negative classes. AUC score of 0 means that the classifier distinguishes positive classes from negative and vice versa.

Precision-Recall Curve Plot: The precision-Recall curve shows the trade-off between precision and recall for different thresholds which is more appropriate than the ROC for imbalanced datasets. As Fig

2.1 shows, the point in the top right shows a perfect classifier while a no-skill(random) classifier will be a horizontal line on the plot with a precision that is proportional to the number of positive examples [2].

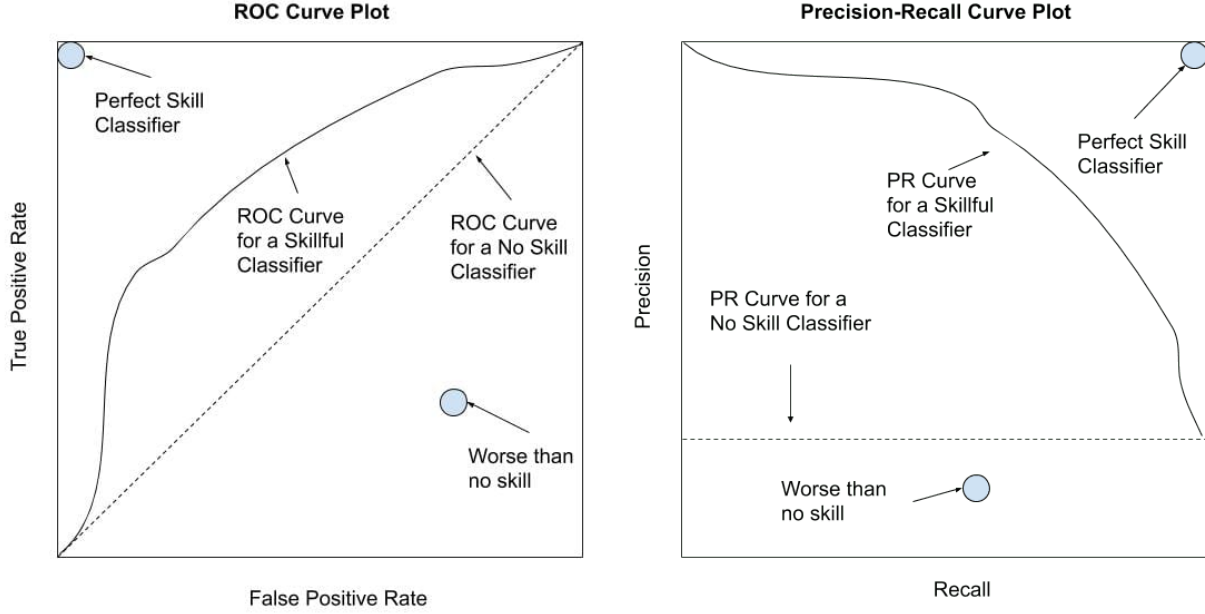


Figure 2.1: Description of ROC and Precision-Recall curve plots in different cases.

MCC: Matthews correlation coefficient (MCC) [81] is a metric for assessing the performance of a classification model which leverages all of the values available in the confusion matrix. It is regarded as a balanced measure that can be used even if the dataset is imbalanced. It is calculated as below formula:

$$MCC = \frac{TP * TN - FP * FN}{\sqrt{(TP + FP) * (TP + FN) * (TN + FP) * (TN + FN)}}$$

The MCC value is between -1 and +1 where +1 represents a perfect prediction, 0 represents a random prediction (no-skill classifier) and -1 represents an inverse prediction.

2.8 Qualitative Study

Qualitative research relies on data obtained by the researcher from sources such as observation, interviews, questionnaires, etc. Qualitative methods include ethnography, grounded theory, discourse analysis, and interpretative phenomenological analysis. This method was developed in the social sciences to enable researchers for studying social and cultural phenomena [34]. This technique has been used in software-related research and there is an empirical study by Huang et al. [56] where the authors review the usage of this method in polished software-related research.

2.8.1 Grounded Theory

Grounded theory is a systematic methodology that has been largely applied to qualitative research. This methodology consists of the construction of hypotheses and theories through the collection and analysis of data. In this method, researchers start to investigate the collected data and then ideas or concepts become more clear to them. Then, these ideas which are emerged from the data are tagged with codes. These codes should summarize the ideas, and they can be grouped into higher-level concepts or categories. Finally, these created categories become the foundation of a hypothesis.

2.8.2 Card Sorting

Card-Sorting is an approach to structuring multiple terms that is particularly suitable for defining topics and group questions in advance. The researcher writes one concept per card and makes sure that cards are in a random order to guard against sorting bias. During the testing session, participants sort the cards into the different categories that make the most sense to them. There are two common sorting approaches:

Closed: There is a predetermined list of categories that participants should choose from that and put the individual cards into these predetermined categories.

Open: In this method, the participant can create categories of their own. In other words, participants can assign any name they want to the groups they have created with the cards in the stack.

2.9 Representation Learning

Representation learning or feature learning aims to learn the representations of the data so it will be easier to extract useful information than manually extracting features from the data. This technique can be either supervised or unsupervised, and it has many applications including but not limited to Speech Recognition, Object Recognition, Natural Language Processing, and Transfer Learning [22].

2.9.1 Embedding

In the context of machine learning, embedding is a mapping from high-dimensional vectors to relatively low-dimensional vectors. In the context of text data, natural language, or programming language, embedding is a mapping from each word to a vector containing numbers. This mapping makes it easier for machine learning algorithms to learn from these vectors for performing the specific required tasks such as classification or generation. The recent embedding models capture the semantic relationship between words well and put the semantically similar words near together in the embedding space, so in this way, each vector can represent

the word and its context as well.

2.9.2 Word Embedding

Word2Vec

Word2Vec [84] represents each word with a particular vector containing numbers that has been used widely in the natural language downstream tasks. In this approach, vectors capture both syntactical and semantically similarity of words. For example, by leveraging the relationship between vectors we can find that

$$\text{vector}(\text{"Queen"}) \approx \text{vector}(\text{"King"}) - \text{vector}(\text{"Man"}) + \text{vector}(\text{"Woman"})$$

. There are two popular algorithms for implementing Word2vec, Continuous Bag-Of-Words and Continuous Skip-Gram. The former approach learns the embedding by predicting the current word given its context while the latter approach learns to predict the surrounding words by having a current word.

Doc2Vec

Doc2Vec [69], an adaption of Word2Vec, is an unsupervised algorithm that provides a feature vector for every document regardless of its length such that the representation of the document should be quite good to predict the words in the document. This algorithm can be used for various tasks such as finding similarities between sentences, paragraphs, or documents.

BERT

BERT (Bidirectional Encoder Representations from Transformers) [35] model which is a deep unsupervised language representation model that learns contextual representations from unstructured text. It was pre-trained on two tasks, Masked Language Modelling (MLM) and Next Sentence Prediction (NSP). In the MLM task, 15% of tokens were masked and the BERT model was trained to predict the masked tokens from their context in the text. For the NSP task, BERT was asked to predict if the second sentence in the provided pair is the subsequent sentence of the first sentence of the pair in the original document.

This model considers the context in which words appear, unlike the Word2Vec model, which improves the efficacy since many words may express different meanings based on the context they are used. For example, the Word2Vec approach generates a single vector for the “apple” word in every context, but the BERT generates different vectors based on its context. One vector would be similar to vectors of words such as “orange” or “banana” and another vector would be similar to vectors of words such as “company” or

“iPhone”.

BERT also considers the position of each word in the sentence unlike the Word2Vec approach before calculating the embedding. Furthermore, it supports the out-of-vocabulary (OOV) by learning at the “Word-Pieces” or “Subword” level which helps to generate embedding vectors for words that it never learns which is a significant improvement over the Word2Vec model which cannot generate any vector for new words it has never seen during the training phase.

2.9.3 Code Embedding

There have been embedding models to leverage the source code such as Code2Vec by Alon et al. [14]. This neural network model is trained on method body code to generate an embedding vector of the source code and then is evaluated by predicting the name of methods. They leverage Abstract Syntax Trees of the code to create path representations. Nguyen et al. [90] use the Word2Vec model to represent the APIs and leverage this representation for automated code translation between Java and C#.NET programming languages. Harer et al. [49] use the Word2Vec model for C/C++ method calls, and then feed this representation to a classification model in order to predict bugs.

CodeBERT

CodeBERT [38] is a bimodal pre-trained model which supports both programming language and natural language. It learns representations of data and supports downstream both natural language and programming language applications such as generating documentation from the code or searching for the most semantically related code from a collection of codes by having the natural language as input. It follows the BERT [35] and RoBERTa [72] models architecture and uses a multi-layer bidirectional Transformer [127]. CodeBERT has the same model architecture as RoBERTa-base with 125M parameters. It has been trained on 6.4M NL-PL pairs of 6 programming languages such as Python, Java, JavaScript, PHP, Ruby, and Go from CodeSearchNet dataset [57]. This model has been trained regarding two objectives, masked language modeling (MLM) and replaced token detection (RTD). The input format follows the concatenation of two segments (first part containing natural language, and the second part containing code) with a separator token: $[CLS], w_1, w_2, \dots, w_n, [SEP], c_1, c_2, \dots, c_m, [EOS]$. The output includes contextual vector representation of both segments (NL and PL), and $[CLS]$ representation can be used as the aggregated sequence representation for various tasks such as ranking or classification purposes.

CodeBERT has been used for different software engineering tasks such as [79] where authors use the CodeBERT model to generate fixes for simple Java bugs with accuracy in the range of 19-72%. Pan et al. [95]

leveraged CodeBERT for bug detection in cross-version and cross-project settings and they also investigated the effects of different prediction patterns. The authors mentioned that they got better performance and less time, and also stated that using sentence-based and keyword-based prediction patterns could improve the prediction performance. Zhang et al. [138] used CodeBERT for enhancing the performance of the question title generation task by using bi-modal information that exists in the question body. They proposed that this method outperforms the previous models under different settings in the range of 4%-14%.

Chapter 3

Proposed Approach

In this section, we provide the overall approach of our study to investigate source code metrics, static analysis tools, machine learning models, and code representation in predicting bug severity. Also, we provide the dataset collection step overview and required pre-processing, but more details of data pre-processing for each RQ are provided in the respective section.

3.1 The Study Architecture

The goal of this study is to investigate bug characteristics with different severity levels. We also investigate different techniques for predicting the bug severity level. For this purpose, we first gather datasets from real-world bugs of two popular datasets named “Defects4J” and “Bugs.jar”. Since the severity label of bugs in these datasets is not provided by the researchers we perform a dataset collection step to fetch all the required data and also any processing steps required for our study regarding each research question. The dataset collection step is explained in Section 3.2.

We provide the overall approach of our study in Fig 3.1. The inputs of our approach are buggy method code in the Java programming language. The first step of our study is to perform the preprocessing step such as removing code comments (to prevent their effects on the result of our experiments) and removing duplicate instances. Then, we extract 10 source code metrics that show promising results in the previous literature for the defect detection task. These metrics are calculated statically from the source code and consist of Lines of Code, McCabe, McClure, Nested Block Depth, Proxy Indentation, FanOut, Readability, Difficulty, Effort, and Maintainability Index which are explained in detail in Section 2.6.

To make sure that these metrics are quite powerful in finding buggyness of code in our dataset regardless of their good results in the previous works, we leverage the Wilcoxon Rank-sum test to find if these metrics

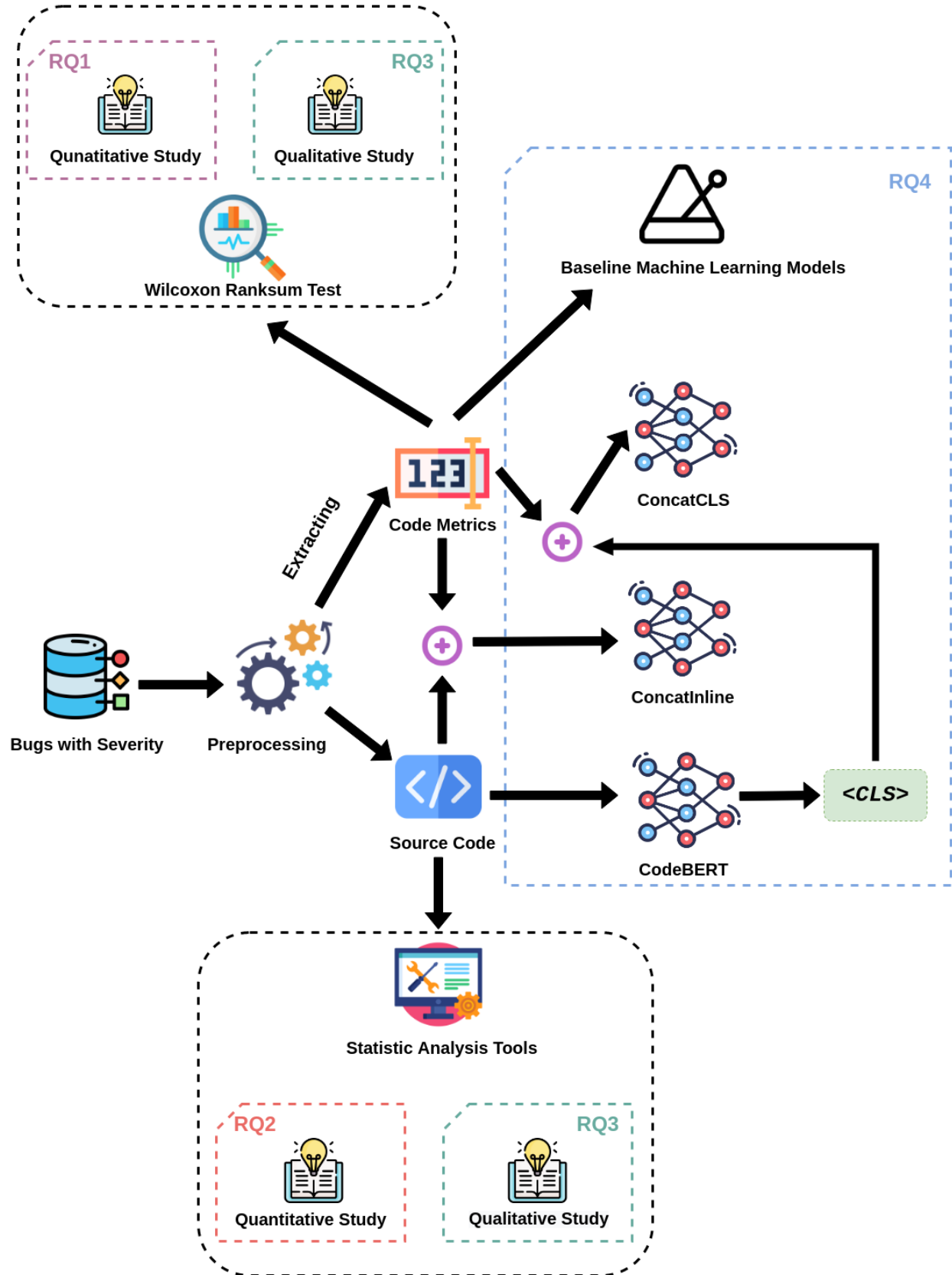


Figure 3.1: The overall approach for our study on bug severity prediction.

can distinguish between buggy codes and non-buggy codes. After finding encouraging results, we applied the same Wilcoxon Rank-sum test to find if these metrics can distinguish between bugs with different severity labels. We performed this experiment by considering the aggregated dataset (all of the samples together from each dataset) and per project. The experiments are provided in detail in Section 4.2.1.

Then we investigate the capability of static analysis tools named “SpotBugs” and “Infer” in estimating buggyness and also bug severity. These tools work on source code by applying different patterns to detect the bug and its severity. The details about the internal logic of these tools are provided in Section 2.5. We provided the reasons behind choosing these tools and an explanation on how we applied them on our problem in Section 4.2.2.

We also perform a qualitative study to find the bug characteristics when the code metrics and static analysis tools cannot detect bug severity levels. For this purpose, we perform a Card Sorting approach, with the help of different people in a few rounds to categorize the type of the bug whenever there is a contradiction between the bug code metrics and its severity label. Also, we sample and manually analyze 25 severe bugs which are totally missed by SpotBugs and also 25 severe bugs which are mislabeled by the SpotBugs tool as low-severity. Finally, we investigate if code metrics and static analysis tools can be used together to increase the overall efficacy of these approaches. The design of each experiment and its results are mentioned in detail in Section 4.2.3.

As a final step and in the RQ4, we use 8 machine learning models as a baseline by training them on the source code metrics, extracted previously. We provide different evaluation metrics for all of the models, which are explained in Section 2.7. Finally, we leverage the source code representation, solely, by using CodeBERT pre-trained model to predict the bug severity level. Also, we integrate the source code metrics to the CodeBERT models, using the ConcatInline and ConcatCLS architectures. In the former version, we concatenate the code metrics with the source code in the textual format, and in the latter architecture, we concatenate the CLS token provided by the CodeBERT model and our vector containing each sample in the dataset. We provide the details regarding the code representation and the CodeBERT model in Section 2.9. Furthermore, our design for this RQ is provided in Section 4.2.4.

3.2 Dataset Collection

Our study requires source code snippet pairs (containing the buggy code and its corresponding fixed code at the method level) along with the severity labels, per code snippet pair. We found that there is no such dataset available, publicly. Therefore, we propose a method to gather the dataset, which is provided in Fig 3.2. The reason behind the decision for choosing Defects4J and Bugs.Jar datasets and also their properties

are mentioned in Section 4.1.

First, we find the corresponding issue for each bug in these datasets from different issue management services such as SourceForge, Jira, and Google Code Archive as shown in Step1. Then, we extract the severity labels which we called Raw Severity Label (RSL).

In step2, we unify Defects4J Raw Severity Labels to create Unified Severity Labels (USLs) according to the provided table for RSLs to USLs mapping, since it contains both numerical and categorical labels. USLs are used for RQ1, RQ2, and RQ3 and we study these research questions for each dataset separately. Our goal was to study bug characteristics in each dataset since these datasets contain different projects with different scopes and sizes, so merging them would affect our study. We provide the details about how we consider these datasets for each RQ in Section 4.2.

In RQ4, we merged these two datasets as shown in Step3 for two reasons. First, we need a relatively large number of samples, so by merging datasets we concluded with enough samples and we did not have to build two separate models. The second reason is our pre-trained model called CodeBERT is trained on millions of samples and it has probably enough generalization capability to work on the merged dataset.

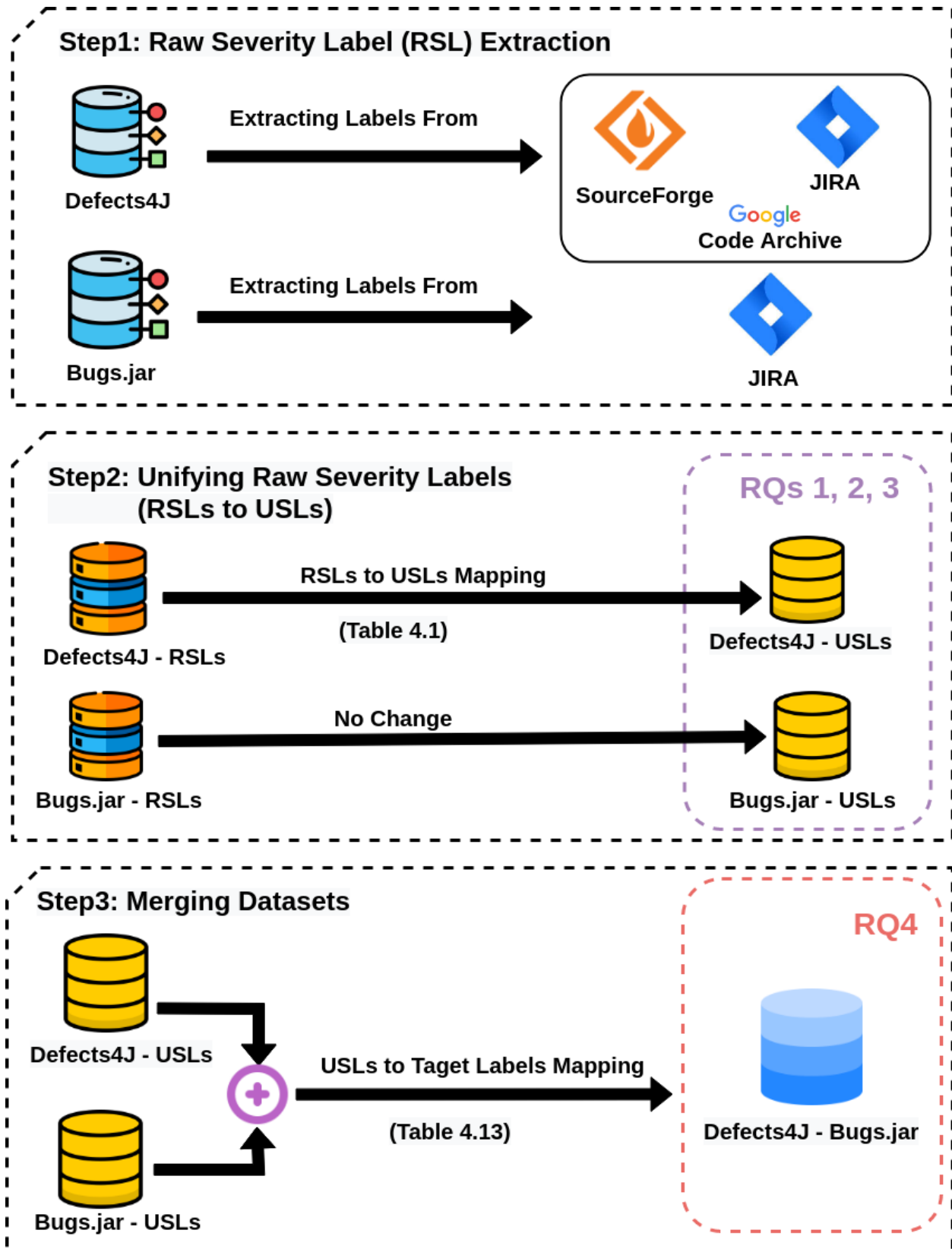


Figure 3.2: Dataset collection approach and required changes for each RQ.

Chapter 4

Empirical Evaluation

In this section, we describe the objectives of our study, the design, and results of our experiments.

4.1 Data Collection

We use Defects4J [59] and Bugs.jar [107] datasets for our study which contain bugs from different popular Java projects. Defects4J is used because it has been widely used in different automated software engineering research domains such as Test Generation [110], Program Repair [77], and Fault Localization [98]. Also, this is a generic dataset containing real bugs from different projects with different domains. Bugs.jar has also been widely used in Program Repair [108] and contains many real bugs and also there is a corresponding bug report for each sample in Jira¹ containing the bug severity. We observed that these datasets contain complex and simple projects which may have different numbers of bugs with different bug severity distributions. For example, in the Defects4J dataset, the “CSV” project is a small and simple project with a limited scope of writing and reading CSV files. It has only 14 bugs, but the Math project which is a very large and complicated library contains 132 bugs. For handling this issue, we perform our experiments both on the whole datasets and each project separately in this study except for the RQ4, and the reason behind this decision is explained in the RQ4 in detail.

4.1.1 Defects4J

The latest version (2.0.0) of Defects4J contains 835 bugs from 17 Java projects. The dataset does not have any information regarding the bug severity itself, so we extract bug severity values from different issue tracking systems. There are three software hosting services used for these projects: Jira, SourceForge, and

¹<https://www.atlassian.com/software/jira>

Table 4.1: Unifying the raw severity labels of Defects4J dataset. For example, the third row means that bugs having raw severity labels of Medium or 5 are considered as Medium bugs.

Unified Severity Label (USL)	Raw Severity Label (RSL)
Critical	Critical
High	High, Major, 3
Medium	Medium, 5
Low	Low, Trivial, Minor, 7, 8, 9

Google Code Archive. After extracting the severity values from these systems, we concluded with 510 bugs out of 835 bugs containing severity labels. Our studied projects of this dataset are Chart, Cli, Closure, Codec, Collections, Compress, Csv, JXPath, Lang, Math, and Time.

After extracting the raw severity labels (RSL), we found that there are many inconsistencies between these raw severity labels of the Defects4J dataset. Some projects use numerical (smaller values indicate higher severity while larger values indicate lower severity) and others use categorical values because of different issue tracking systems. Therefore, we unified similar raw labels to produce four meaningful unified severity labels (USL): “Critical”, “High”, “Medium”, and “Low” severity labels. Table 4.1 shows the mapping between all RSL and the USL labels of the Defects4J dataset.

4.1.2 Bugs.jar

The Bugs.jar dataset contains 1,158 bugs from eight large, popular, and diverse open-source Java projects. These projects are accumulo, camel, commons-math, flink, jackrabbit-oak, logging-log4j2, maven, and wicket. All of these projects use Jira as an issue management system, and each of the bugs has an assigned severity level. The severity labels from this dataset are “Blocker”, “Critical”, “Major”, “Minor”, and “Trivial”. Since the RSLs of this dataset are consistent, there is no need for a unifying process like what we did for the Defects4J dataset, and we considered all the RSLs as USLs

After mentioned preprocessing step we concluded with two datasets containing the USLs (RSLs are discarded and not used anymore). The Defects4J dataset contains “Critical”, “High”, “Medium”, and “Low”. Bugs.jar dataset contains “Blocker”, “Critical”, “Major”, “Minor”, and “Trivial”. This is our extracted and preprocessed dataset, but we will group some of these USLs in different RQs according to their experiment design requirements, which will be explained in their related sections accordingly.

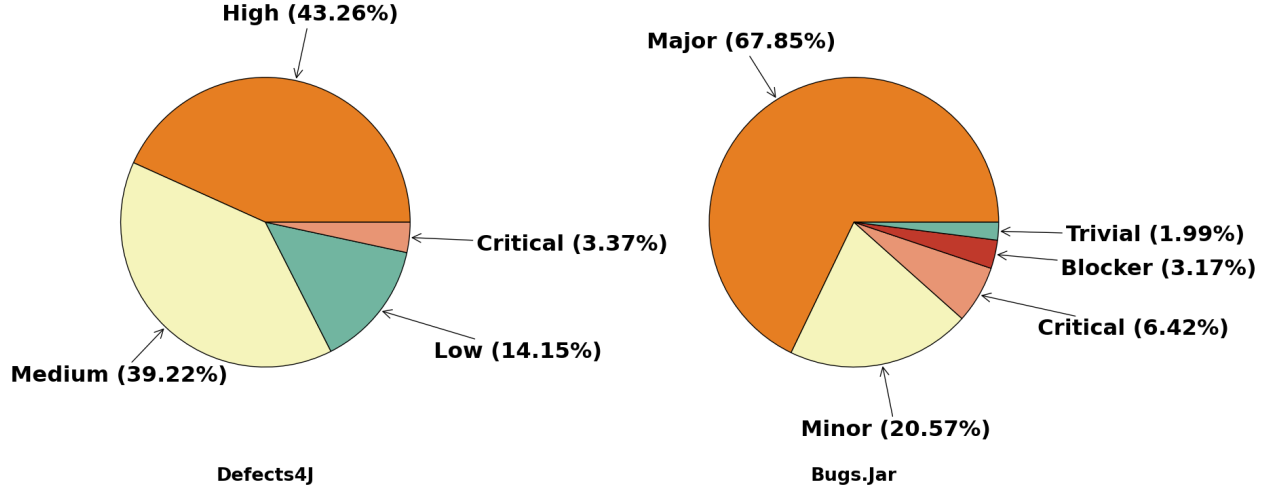


Figure 4.1: Buggy methods severity distributions of Defects4J and Bugs.jar datasets with their USL values.

4.1.3 Buggy vs. Not Buggy

Since we focus on the method-level granularity, we consider a method as a buggy method if it is modified/removed by a bug fixing patch. If a method, however, is introduced with a bug fix patch, we do not label this method as buggy. We discard the static initialization blocks and constructors since those are special types of methods that are used for initializing the enclosed class, mostly.

A bug fixing patch, however, can impact multiple methods, in that case, we consider all of them as buggy. This has been a common practice in earlier studies (e.g., [31, 97, 87]). We discuss the relevant threats in Section 4.4. Finally, from the 510 bugs of the Defects4J dataset, we found 742 buggy methods and from the 1,158 bugs of the Bugs.jar dataset, we found 2616 buggy methods.

Similar to earlier studies (e.g., [31, 97, 87]), we considered a method as non-buggy if it was not modified in the current bug fixing patch inside the buggy class. In this way, we are extracting methods that are contributing to the same functionality as the buggy methods since methods of a class have high cohesion.

We extracted 20,179 and 57,197 non-buggy methods from the Defects4J and Bugs.jar, respectively. The list of the buggy and non-buggy methods containing their project name, class name, start line, end line, and bug severity (corresponding bug severity for non-buggy methods) is available in our publicly shared GitHub repository ²

The severity distributions of the buggy methods for Defects4J and Bugs.jar datasets are shown in Fig 4.1. We study these two datasets separately because of the difference in their labels. The severity distribution per project is shown in Fig 4.2 and Fig 4.3. The figures show that some projects do not have enough samples in all severity categories such as **Chart** project, containing only a few bugs in “Medium” and “Low” severity

²<https://github.com/EhsanMashhadi/EMSE-BugSeverity>

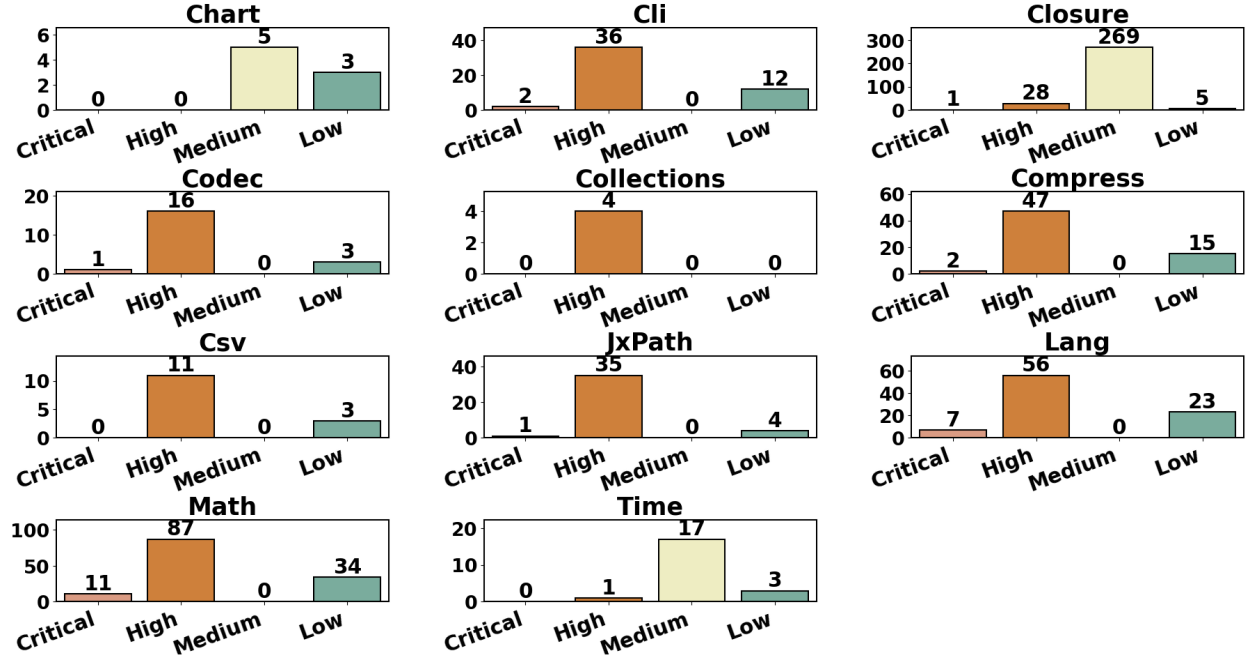


Figure 4.2: Buggy methods severity distributions in Defects4J dataset with the USL values.

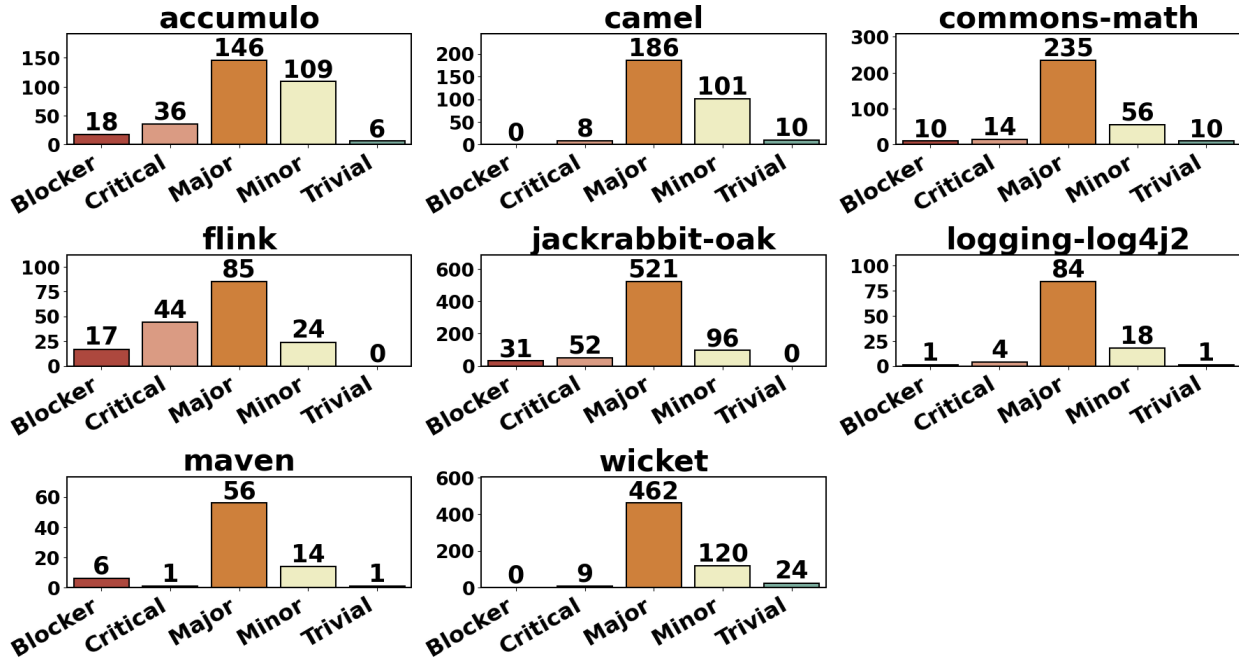


Figure 4.3: Buggy methods severity distributions in Bugs.jar dataset with the USL values.

groups, or the **Closure** project, which mainly contains Medium severity bugs. Because of not having enough samples in each project/severity group, we may combine data in different RQs, which we discuss later.

4.2 Experiment Design and Results

In this section, we provide the motivation, experiment designs, and results for each RQ. Since each RQ has a few sub-RQs they may have different experiment designs which are explained in the related section.

4.2.1 Code Metrics & Bug Severity (RQ1)

We aim to understand the capabilities of source code metrics in finding bugs and predicting their severity labels.

Motivation

One of the most important attributes in every issue tracking system is bug severity which is usually determined manually by the developers/QA team. This process is time-consuming and error-prone since the technical team should investigate the effects of the reported bug (e.g., number of affected users, number of crashes, probable consequence on the whole system) based on the bug description or other available data from analytical systems (e.g., crash report logging systems).

As we have mentioned in Section 2, there have been various prosperous research that uses source code metrics for defect-related issues, but the severity prediction studies mostly use bug description which works in cases where there is a well-written bug description and also is quite subjective. Therefore, in this RQ we study source code metrics' ability in predicting bug severity.

Approach

We examine the code metrics in predicting buggyness and the severity labels by following RQ and sub-RQs.

RQ1: Are source code metrics good indicators of buggyness and bug severity?

RQ1-1: Do source code metrics identify buggy code?

RQ1-2: Do source code metrics distinguish between different bug severity?

Design of RQ1: In this RQ, we first assess code metrics' capabilities in finding bugs and then evaluate them further in terms of their potential in estimating the bugs' severity, using a static analysis test. To find

the answer to this RQ, we apply a statistical test to see if distributions of methods (e.g., buggy vs non-buggy or critical vs low) are statistically different according to our selected code metrics.

After applying the Shapiro-Wilk test [111] using 5% level of significance ($\alpha = 0.05$), we found out that none of our distributions are normal, so we use the non-parametric Wilcoxon Rank-sum test for answering RQ1 with the 5% level of significance ($\alpha = 0.05$) with the following two null-hypotheses for RQ1-1 and RQ1-2:

- RQ1-1: Distributions of buggy methods and non-buggy methods are not statistically different.
- RQ1-2: Distributions of buggy methods with different severity values are not statistically different.

For finding the significance of the difference, we calculated Cliff’s Delta effect size [32]. We followed the values provided by Hess and Kromrey [53] for interpreting the result of this value. According to the mentioned study, researchers consider values smaller than 0.147 to be negligible, values in the range [0.147, 0.33) to be small, values in the range [0.33, 0.474) to be medium, values greater than 0.474 to be large.

Since we are using the static analysis test for this RQ, it is important to make sure that each distribution has enough samples, and distributions are diverse enough (having samples from different projects for reducing the bias problem) to make the results robust. Also, we discard the methods with $LC < 4$ in this RQ since they are mostly boilerplate code such as setters, getters, and constructors which are generated automatically.

We found that our studied datasets contain imbalanced distributions for RQ1-2 since some USLs contain only a few samples (e.g., the Critical USL in the Defects4J contains only 25 samples) or some USLs do not contain enough bugs from various projects to make that category diverse (e.g., 92% of samples in the Medium USL of the Defects4J dataset are from `Closure` project). Because of the mentioned problem, we merge some USLs (Defined in Section 4.1) one level further for RQ1-2 which is shown in Table 4.2.

Table 4.2: Merging different USLs to create categories for Defects4J and Bugs.jar datasets. For example, the first row means that bugs having Critical or High USLs are considered in the Critical category.

Dataset	Unified Severity Label (USL)	Merged Category
D4J	Critical, High Low, Medium	High Low
Bugs.jar	Blocker, Critical Major Minor, Trivial	Critical Major Minor

As shown in Table 4.2, for the Defects4J dataset, the Critical USL is merged into the High, and also the Medium and Low USLs are merged, so we ended up with two categories of High and Low severity bugs.

In the Bugs.jar dataset, the Blocker and Critical USLs are merged, and also the Trivial and Minor USLs are merged, so we concluded with three severity categories of Critical, Major, and Minor. The Blocker and Critical USLs are merged since the Critical USL does not contain enough samples, the Minor and Trivial USLs are merged because of not enough sample in Trivial. We keep the Major USL as an independent category since it has enough samples.

Results

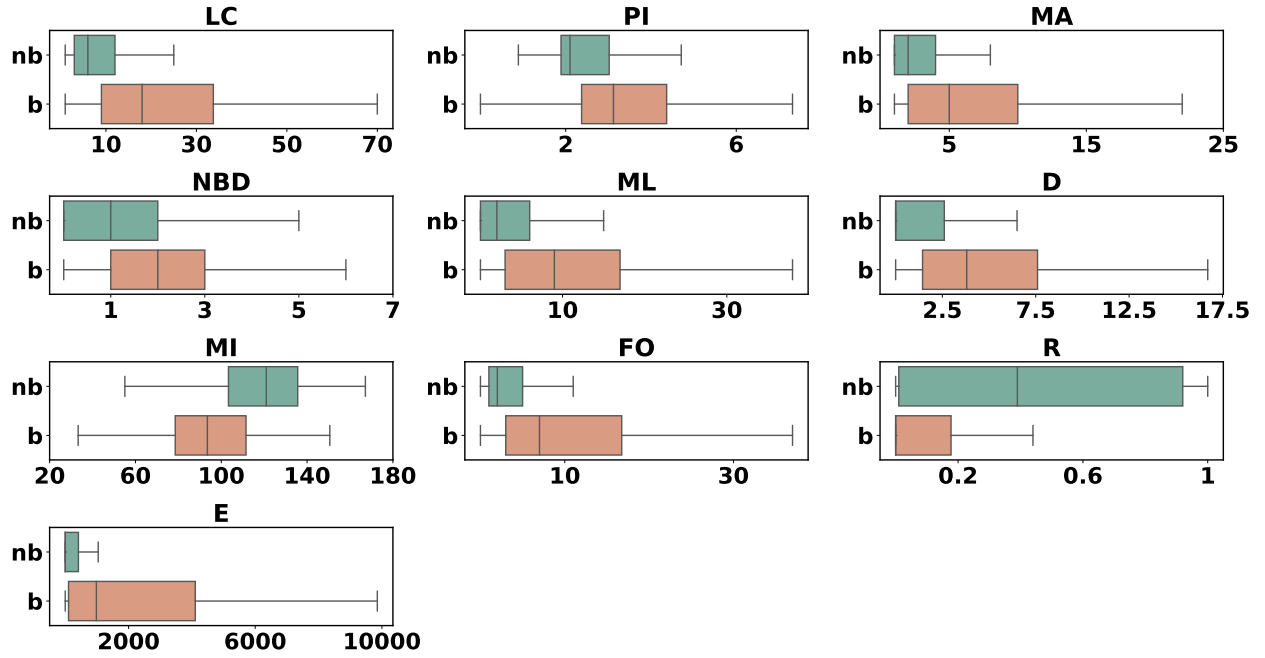
RQ1-1 Answer: Figure 4.4 compares the distribution of different code metrics between buggy and non-buggy methods (Figure 4.4a for the Defects4J, and Figure 4.4b for the Bugs.jar datasets). These are aggregated results, as we have combined data for all the projects in a specific dataset. Results suggest that the median values of all code metrics except for the Readability and Maintainable Index in the buggy methods are larger than the median or Q3 (75th percentile) values in the non-buggy methods. Also, for Maintainable Index and Readability, the median of non-buggy methods are larger than the median or Q3 (75th percentile) values in the non-buggy methods. Evidently, the buggy methods are generally larger (LC), more complex (e.g., McCabe, McClure), and less readable than the non-buggy methods. These figures imply that buggy and non-buggy distributions are different regarding all code metrics.

After applying the Wilcoxon Rank Sum Test to find if there is a statistical difference between these two distributions, we found that all of our comparisons for code metrics between buggy and non-buggy methods are statistically significant ($P \leq 0.5$). Therefore, the null hypothesis is rejected which means that these metrics are able to distinguish between buggy code and non-buggy code. We provided the Cliff’s delta size values in Table 4.3 which shows that effect size values are small or medium in all cases. In particular, LC, FanOut, Maintainable Index and, Halstead Effort exhibit better performance than other metrics, because for them the effect size is medium in both datasets.

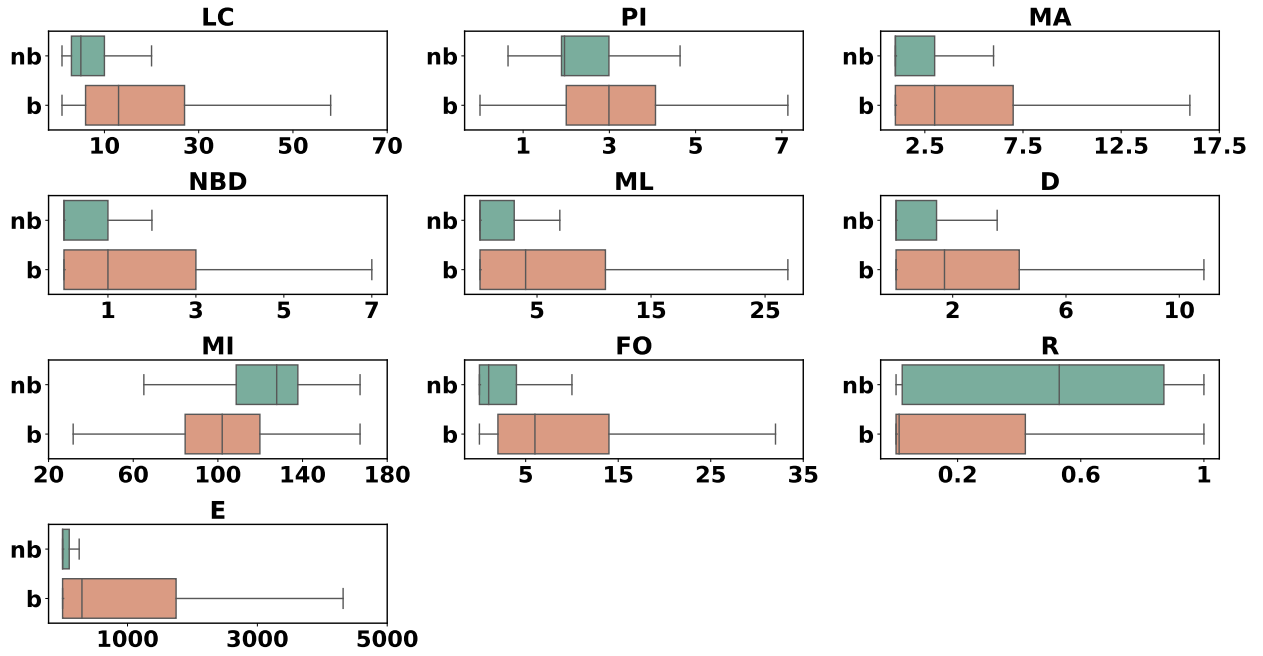
Table 4.3: Cliff’s Delta Effect sizes for comparing code metrics between buggy and non-buggy methods. S refers to Small and M refers to Medium effect size.

Dataset	LC	PI	MA	NBD	ML	D	MI	FO	R	E
Defects4J	M	S	M	S	M	S	M	M	S	M
Bugs.jar	M	S	S	S	S	S	M	M	S	M

With aggregated analysis, however, we can not observe if our results are true for all of the projects. The aggregated data can be highly influenced by a few large projects. Also, different external factors, such as code review policy [131], developers’ commit patterns [52] and expertise [80], can impact the distribution of



(a) Defects4J



(b) Bugs.jar

Figure 4.4: Comparing source code metrics between buggy methods (b axis) and non-buggy methods (nb axis) using aggregated dataset.

code metrics and bug-proneness of a particular project. Therefore, we now reproduce the results for each project individually.

Table 4.4: Wilcoxon Rank-sum test results to show the percentage of projects where code metrics exhibit statistically difference between buggy and non-buggy methods.

Dataset	LC	PI	MA	NBD	ML	D	MI	FO	R	E
Defects4J	100%	50%	70%	80%	80%	70%	100%	100%	90%	80%
Bugs.jar	100%	87.5%	100%	100%	100%	100%	100%	100%	100%	100%

Table 4.5: Cliff’s Delta Effect sizes after comparing buggy vs non-buggy methods. N refers to Negligible, S refers to Small, M refers to Medium, and L refers to Large effect size. For example, the first column of the first row compares the LC distributions between buggy and non-buggy methods and shows the percent of differences with negligible, small, medium, and large effect sizes.

Metrics	Defects4J				Bugs.jar			
	N	S	M	L	N	S	M	L
LC	0	40	30	30	0.0	25	62.5	12.5
PI	30	50	10	10	37.5	62.5	0.0	0.0
MA	20	40	20	20	12.5	37.5	50	0.0
NBD	20	50	10	20	12.5	75	12.5	0.0
ML	10	30	40	20	0.0	62.5	37.5	0.0
D	20	40	30	10	12.5	37.5	50	0.0
MI	0	30	40	30	0.0	25	62.5	12.5
FO	0	40	40	20	0.0	37.5	50	12.5
R	0	50	40	10	0.0	87.5	12.5	0.0
E	10	30	30	30	0.0	37.5	62.5	0.0

We calculated the percent of projects where the distributions of a particular metric are statistically different between buggy and non-buggy methods. Table 4.4 shows the results for all the code metrics. The distribution of LC, Maintainable Index, and FanOut are statistically different between buggy and non-buggy methods in all the studied projects from both datasets (100%). In the Defects4J dataset, the proxy indentation (PI) metric has the weakest performance since in 50% of the projects, the result is not significantly different. Most of the metrics, however, show good performance in distinguishing the buggy method from the non-buggy method. We also calculated the Cliff’s effect sizes of the differences, as presented and described in Table 4.5. The results show that Lines of Code, Maintainable Index, Fan Out, and Readability metrics do not exhibit negligible effect size for both datasets, but Proxy Indentation has the largest percentage of negligible effect size in both datasets.

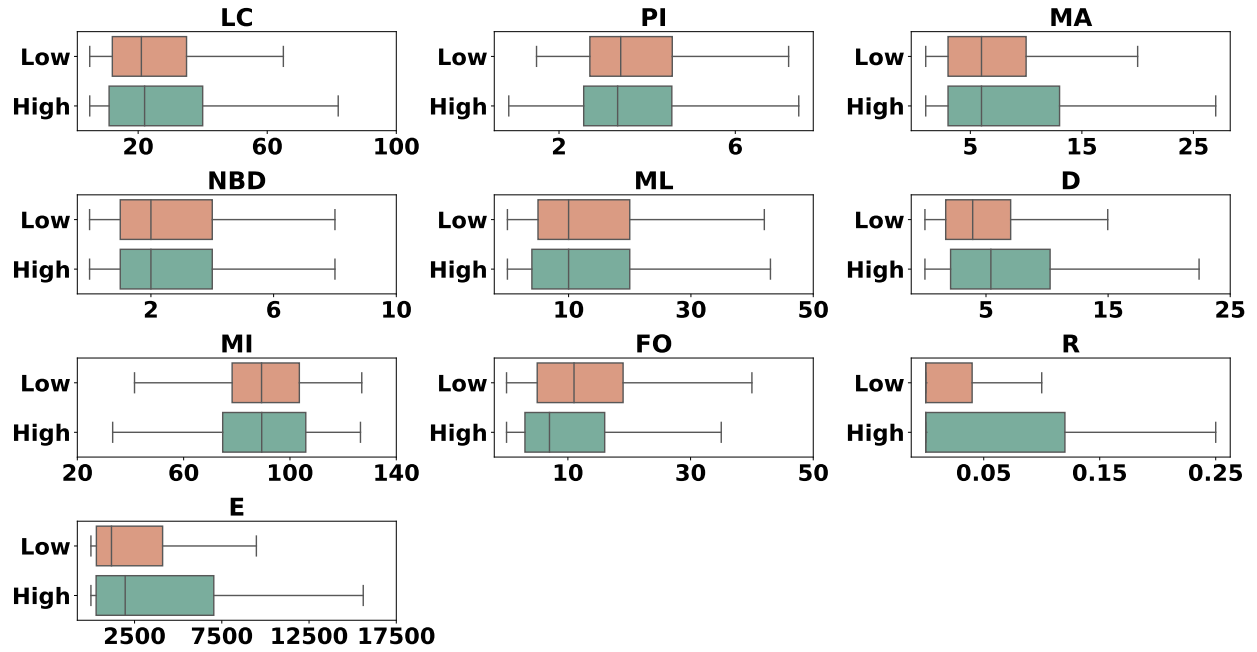
Summary of RQ1.1 Results: Our selected code metrics show high performance in identifying the buggy-ness of methods in both datasets, and on both the aggregated datasets and the individual projects. Line of Code, Maintainable Index, Fan-out, and Effort metrics have the best performance while Proxy Indentation and Nested Block Depth have the worst performance.

RQ1.2 Answer: After observing the somewhat known power of code metrics in distinguishing buggy and non-buggy methods, in this sub-RQ, we show their power in estimating the bug’s severity. Although we wanted to perform the analysis for each project separately, there is not enough data in each severity category, except for `accumulo` and `jackrabbit-oak` projects. The `accumulo` project has 44, 100, 87 and `jackrabbit-oak` project has 71, 439, 80 samples in Critical, Major, and High categories (we described in Section 4.1) respectively. The number of samples for each category in other projects was much lower, so we did not report their statistical test results separately.

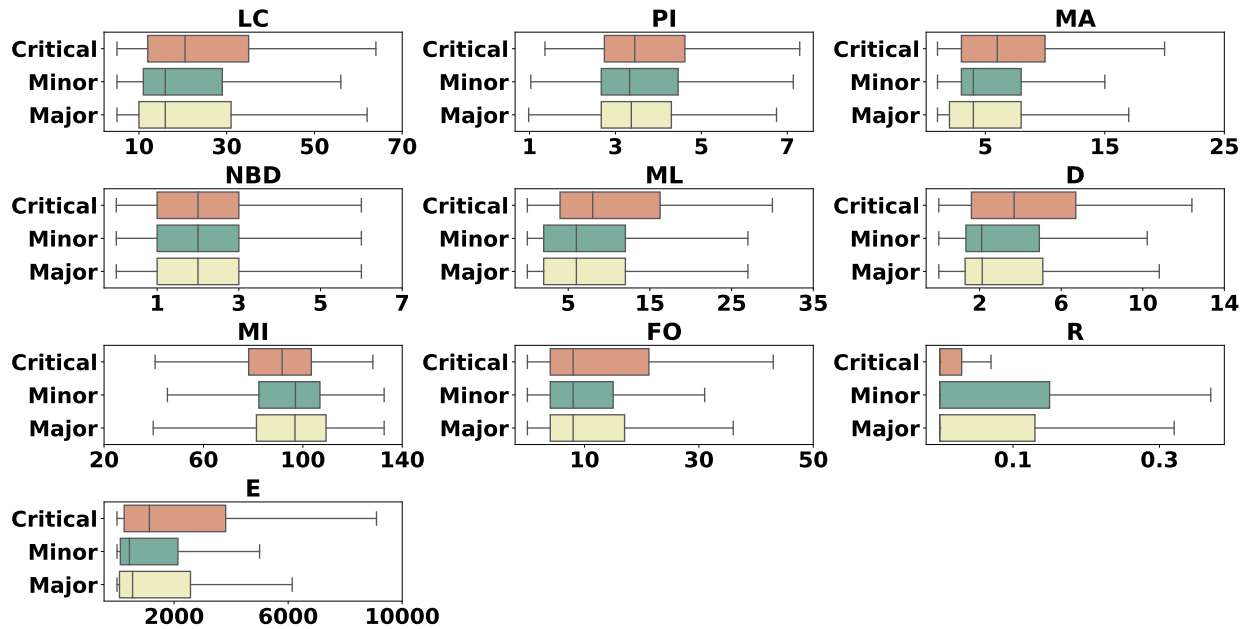
Fig 4.5 compares the distributions of different code metrics after grouping the bugs into different bug severity categories. To provide better insights, we also provide the Wilcoxon Rank-Sum test and Cliff’s Delta size values for both datasets in Table 4.6. While the first two major rows (for Defects4J, and Bugs.jar) show results for aggregated analysis, the last two rows also show results for the two individual projects: `accumulo` and `jackrabbit-oak`.

In contrast to the previous section, in many cases, the results are not significantly different between different bug severity categories. Considering the aggregated analysis in both datasets, only Halstead difficulty, and effort show desired behavior: methods with higher severity bugs have higher difficulty and effort distribution. Maintainability index (MI) performs the worst: only in two cases it exhibits statistically different distributions. Readability shows desired behavior for the Bugs.jar dataset when comparison was done between critical and major, and critical and minor. The statistically different distributions among these groups suggest that, a method with lower readability has higher chance to have more severe bugs. Although source lines of code (LC) was very helpful to differentiate between buggy and non-buggy methods (Figure 4.4b, Table 4.4 and Table 4.5), its performance is significantly worse when it comes to distinguishing different levels of bug severity. This is interesting because numerous previous studies (e.g., [37, 43, 115]) has shown that size (source lines of code) is the most important code metric, and none of the other code metrics provide any new information if their correlation with size is normalized. Our result challenges that claim because we find the superior performance of other code metrics (e.g., Halstead difficulty, and Halstead effort) than size when distinguishing methods with different bug severity.

In many cases, there is no statistical difference between these code metrics when compared against bug



(a) Defects4J



(b) Bugs.jar

Figure 4.5: Comparing source code metrics between different severity groups for Defects4J and Bugs.jar datasets.

Table 4.6: Cliff’s Delta Size for bugs with different severity values. α was set to 0.05. A red cell indicates that the null hypothesis is rejected (i.e., the distributions are statistically significantly different), whereas a blue cell means they are not statistically different. Effect sizes are N~Negligible, and S~Small. For example, the first cell shows that in the Defects4J dataset, the distribution of source lines of code (LC) between methods with high and low severe bugs is not statistically different (blue) and the difference has a negligible effect size.

Project	Distributions	LC	PI	MA	NBD	ML	D	MI	FO	R	E
Defects4J (all)	High-Low	N	N	N	N	N	S	N	S	N	N
Bugs.jar (all)	Critical-Major	N	N	S	N	S	S	N	N	N	S
	Critical-Minor	N	N	N	N	N	S	N	N	N	S
	Major-Minor	N	N	N	N	N	N	N	N	N	N
accumulo	Critical-Major	N	N	S	S	S	N	N	N	N	N
	Critical-Minor	S	S	S	S	S	S	S	S	N	S
	Major-Minor	N	S	N	N	N	N	N	S	N	N
jackrabbit-oak	Critical-Major	N	N	N	N	N	N	N	S	N	N
	Critical-Minor	N	S	N	S	S	S	N	N	S	N
	Major-Minor	S	S	S	S	S	S	N	S	N	S

severity. It means that there are some cases where the code is complex which makes it error-prone, but severe bugs are not necessary in the complex code. We may intuitively say that there may be bugs in a simple method containing a SQL query for reading data from a database. In such a method, the code complexity metrics suggest a simple method, but if the developers do not handle input validation, so the code will be vulnerable to SQL Injection. On the other hand, we may have a very complex and big method containing nested loops with different switch cases and exception handling statements. The code complexity metrics suggest the complex method, but this method may try to handle the UI/GUI part of the system only, which is not vulnerable to any critical or high severity bugs. We will explore the real examples from our datasets regarding our intuition in RQ3.

Summary of RQ1.2 Results: None of the selected code metrics show promising results in distinguishing bug severity. While Difficulty and Effort metrics have the best performance in the aggregated datasets, the Maintainable Index and Proxy indentation metrics have the weakest performance. Although the Line of Code metric has the best performance in distinguishing bugginess of the methods, it has the worst performance in finding bug severity, in both the aggregated datasets and the individual projects.

4.2.2 Static Analysis Tools & Bug Severity (RQ2)

The second goal of our study is to find out if static analysis tools can detect the bugs and their severity labels.

Motivation

Since source code metrics do not show promising results in predicting the bug severity according to Section 4.2.1, we explore other approaches. To do so, we study the static analysis tools which are widely used in practice for different purposes (e.g., finding programming errors, coding standard violations, syntax violations, security vulnerabilities) by technical teams. This approach is favorable because of its higher speed than the human code review and its capability in working offline with low required resources. Since these tools are already integrated into the deployment process of many companies, so if we could use them for predicting the severity we can leverage them for prioritizing the bugs without requiring any other tools/methods. Therefore, in this RQ, first, we need to find out if these tools can detect real-world complex bugs, and then we study their performance in predicting the detected bug’s severity.

Approach

We pose one RQ with its two sub-RQs to find the static analysis tools capabilities in bug/severity detection.

RQ2: What is the capability of static analysis tools in finding bugs and their severity?

RQ2-1: How effective static analysis tools are in detecting buggy methods?

RQ2-2: Can static analysis tools estimate detected bugs’ severity?

Design of RQ2: To answer this RQ, we selected two static analysis tools named SpotBugs, [9] and Infer [6]. We selected SpotBugs because it not only can detect bugs but also report a Rank value which indicates the severity. We used the popular Infer tool only to detect buggy methods regardless of their severity values. Since there is no need for a statistical test in this RQ, and therefore there is no restriction on the number of samples in each group we used the USL values (mentioned in Section 4.1) directly without a need for any merging process.

SpotBugs has different configurations which affect its performance in finding bugs. It has some specific detector modules, which focus on specific bug types, but we used the standard detectors, which is also the default option. Also, this tool has a configuration option named `effort` which adjusts the taken effort in finding bugs. We used the `effort=max`, which is the highest level of effort, so in this way, we have provided as much as the computation cost this tool requires to work on our datasets.

There are different ways to provide projects’ source code to these tools, such as providing only the buggy class(es), the package containing the buggy class(es), or the whole project. By providing only the buggy class, the tool may not be able to find the bugs that are across different classes since it only analyses that

specific class, but when we provide the whole package or the whole project, it analyses any file in these modules to find the bugs accurately which also needs more resources and time. We provided the whole project to be analyzed with tools to make the result more robust. However, this takes 3 and 52 hours for the Defects4J and Bugs.jar dataset, respectively, on a regular computer with 16GB RAM and 16 CPU cores.

Although we successfully applied these tools on all of the Defects4J projects, for several projects of the Bugs.jar dataset we faced build errors due to dependency issues. We were successful in building `flink`, `commons-math`, and `accumulo` projects which consist of 248 buggy methods.

SpotBugs reports the **Confidence** value for each bug instance, which indicates the level of confidence the tool has in reporting this warning. This attribute may have different values such as 1 to match high-confidence warnings, 2 to match normal-confidence warnings, or 3 to match low-confidence warnings. We considered all of the reported warnings regardless of their confidence values.

Also, this tool reports the **Rank** property which indicates the bug rank. This property is an integer value between 1 and 20, where 1 to 4 are scariest, 5 to 9 scary, 10 to 14 troubling, and 15 to 20 of concern bugs. [9]. This property has a similar semantic to the bug severity value, so we used this value to see the tools capability in finding the bugs' severity.

Table 4.7: Translation between the SpotBugs reported rank (SRR) value and the unified severity label (USL) value.

Unified Severity Label (USL)	SpotBugs Reported Rank (SRR)
Critical, Blocker	1 to 4
Major, High	5 to 9
Medium	10 to 14
Low, Trivial, Minor	15 to 20

The translation between the range of this value and our USLs is provided in Table 4.7. As the table shows, we consider 1 to 4 as Critical and Blocker USL, 5 to 9 as Major and High USL, 10 to 14 as Medium USL, and 15 to 20 as Minor, Low, and Trivial USL. For example, when the SpotBugs reports any number in the range of [1,4], and the actual severity value is either Critical or Blocker we say that it detects the severity correctly.

The Infer tool reports the **IssueType** attribute with these categories: 1) Error 2) Warning 3) Info, Advice, Like. These values can be interpreted as the bug importance, but we found that all of our bugs are mapped to the Error category, based on their severity values. Thus it is not possible to leverage the **IssueType** property to predict the bug severity, and we cannot use this tool to predict the severity. Therefore, we leverage this tool only to see if it can detect buggy methods, regardless of their severity values.

Both of these tools report the roots of detected bugs according to their defined patterns, by locating the start and end lines of the bug. Since we are considering bugs at the method-level granularity, we consider a method to be buggy whenever these tools report a bug in that method regardless of the reported start and end lines.

Results

RQ2-1 Answer: After applying SpotBugs and Infer tools we count the number of buggy methods that are reported as buggy (TP), the number of non-buggy methods which are not reported as buggy (TN), the number of buggy methods that are not detected by tools (FN), and the number of non-buggy methods which are reported as buggy methods (FP). If any of these tools report more than one bug for each method we only count that method once to prevent the duplication problem.

Table 4.8: Static analysis tools performance in finding buggy methods.

Dataset	Tool	Accuracy	Precision	Recall	F1
D4J	SpotBugs	93%	7.1%	7.2%	7.1%
	Infer	95%	9%	1.9%	3.1%
Bugs.jar	SpotBugs	95%	7.6%	4.3%	5.4%

We calculated the Accuracy, Precision, Recall, and F1 values which are shown in Table 4.8. These values show that these tools have poor performance in detecting buggy methods of both datasets. The precision value range is 7%-9% which means that they may report many non-buggy methods as buggy methods, so the technical team may spend extra time on finding bugs in these methods, and this increase the cost of the development phase without any useful result. However, the recall value (predicting buggy methods as non-buggy methods) is a more important metric in our case which has a smaller range of 2%-7%. This small value indicates that these tools miss many bugs which may lead to harmful consequences. The calculated F1 score shows that SpotBugs have better performance in finding bugs in the Defects4J dataset than the Bugs.jar dataset, and Infer tool has worse performance than SpotBugs for the Defects4J dataset.

Our results are consistent with the previous study [45] in 2018 where the authors studied the capability of SpotBugs, Infer, and ErrorProne tools in finding buggy methods of an older version of the Defects4J dataset. They concluded that 95.5% of buggy methods are not detected by these tools. Surprisingly, although these tools have been updated several times since four years ago, their performance in finding bugs is not improved much.

Since these tools are matching the provided code with their predefined generic bug patterns and from the

poor obtained performance we can say that many bugs are missed by these tools. However, we will perform a qualitative study of some randomly sampled bugs to find the reasons behind this poor performance, in RQ3.

Summary of RQ2.1 Results: Both SpotBugs and Infer tools have significantly low performance in finding bugs even with many existing developed patterns. The precision range for both datasets is 7%-9% and the Recall range is 2%-7% which shows the high number of missed bugs.

RQ2.2 Answer: Since only the SpotBugs tool can detect the severity of bugs, in this section we provide the results of only this tool on both Defects4J and Bugs.jar datasets.

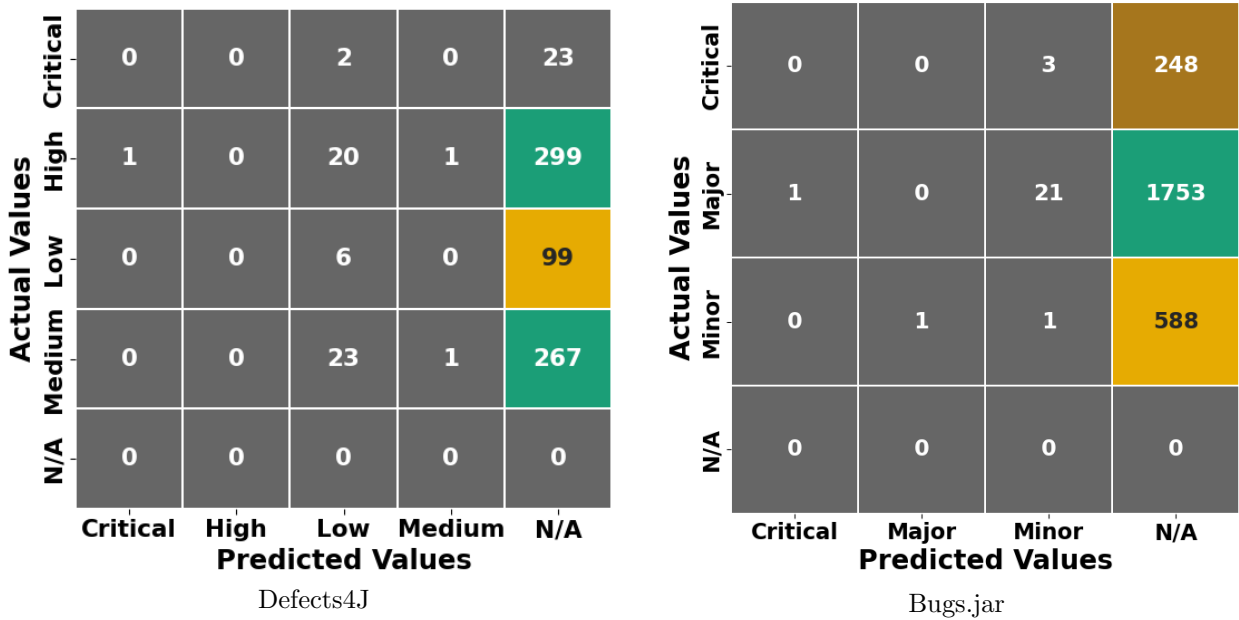


Figure 4.6: Confusion matrices of bug severity prediction by SpotBugs for Defects4J and Bugs.jar datasets. N/A means that the tool does not predict a value. For example, in the Defects4J dataset, 23 critical bugs have not been detected. Colors show the magnitude of numbers.

We provided the confusion matrices for SpotBugs tool in Fig 4.6. These figures show that this tool cannot detect bug severity in many cases, and also mislabeled many bugs with wrong severity labels. For example, it detects two critical bugs as low-severity bugs and 20 high-severity bugs as low-severity bugs, in the Defects4J dataset. These figures show that the SpotBugs did not detect the severity of any Critical bugs in Defects4J and Bugs.jar datasets.

Table 4.9 shows different descriptive statistics of this tool performance. Since we have multi-class labels, we used **macro average** [7] for calculating these values. In this method, the final result is computed by taking the arithmetic mean of all the per-class descriptive scores.

The overall score F1 score shows that SpotBugs has significantly better performance for the Defects4J dataset, but the scores show that this tool has, overall, a very low performance. We studied bug characteristics manually in RQ3 to find the reasons for this weak performance.

Table 4.9: SpotBugs performance in finding bug severity of Defects4J and Bugs.jar datasets.

Dataset	Accuracy	Precision	Recall	F1
D4J	94.00%	12.35%	1.25%	1.67%
Bugs.jar	0.03%	1.00%	0.04%	0.08%

Summary of RQ2.2 Results: SpotBugs has significantly better performance in finding bug severity of the Defects4J dataset than the Bugs.jar dataset. However, this tool wrongly assigns lower severity values to bugs in many cases. The overall performance shows that there is still much work to be done for this tool to accurately estimate bug severity.

4.2.3 Manual Analysis of Bug Characteristics (RQ3)

Our final goal, in RQ3, is to dig more into the results of RQ1 and RQ2 and study the bug characteristics in different important scenarios such as (a) when there is a contradiction between code metrics and severity, (b) when the static analysis tools miss/mislabel severe bugs, and (c) when code metrics and static analysis tools can be complementary.

Motivation

To understand why source code metrics cannot indicate the bug severity, we need to study the cases where there is a contradiction between the bug severity and source code metrics. Also, it seems important to find out if there is any relationship between the bug’s type and its severity. The answer to this question can help researchers to consider bug types as important features when providing new approaches for bug prediction to have better accuracy, and also developers may pay more attention to specific bug types during the manual severity assignment process.

To find out why static analysis tools miss many severe bugs or predict lower severity than the actual severity, in most cases, we need to study if this problem exists because of the nature of static analysis tools or if there is potential for improvement. The answer to this question will help researchers to improve the developed approaches to enhance the bug severity performance and also developers can get insights into how confidently they should use these tools in their projects.

Finally, since real bugs are very complex due to the complexity of the current software and various new

technologies, it would be useful to consider multiple approaches instead of relying on a single approach to predict the severity. Therefore, we also study if code metrics and static analysis tools are complementary in terms of bug severity prediction. This may lead to a technique that combines these approaches to predict the severity more accurately.

Approach

We provided one RQ with three sub-RQs to study the bug characteristics.

RQ3: What are the characteristics of bugs with different severity values?

RQ3-1: What factors may lead to inconsistencies between bug severity and code metrics?

RQ3-2: Why do static analysis tools miss or mislabel severe bugs?

RQ3-3: Do the results of code metrics and static analysis tools have overlap?

Design of RQ3-1: As we show in the RQ1 results section, there is a correlation between bug existence and code complexity metrics, but there is no significant correlation between bug severity and code complexity metrics. Therefore, we manually investigated a random sample of bugs for finding factors that lead to contradictions between bug severity values and code metrics. The goal of this sub-RQ is to study the characteristics of buggy methods with high severity (e.g., Blocker, Critical, Major) which have low code metrics measurement. Also, to study buggy methods with low severity (e.g., Low, Minor, Trivial) which have high code metrics measurement.

To facilitate the manual analysis, we split all bugs into two high-level categories of severity, which are: severe (including Blocker, Critical, Major, and High severity USLs from RQ1) and non-severe (including Medium, Low, Minor, and Trivial USLs from RQ1).

To find the two lists of the most and the least complex methods, we calculate a representative sum complexity value from different code metrics. Since our code metric values do not have the same range, we normalize them by using the Robust Scaler algorithm [8], which is robust against outliers. It follows a similar algorithm to the MinMax scale, but it uses the interquartile range rather than the min-max. The formula for Robust Scaler is:

$$\frac{x_i - Q_1(x)}{Q_3(x) - Q_1(x)}$$

Where Q_1 is the 1st quartile, and Q_3 is the third quartile.

To find the Sum Complexity (SC) value—which indicates the total complexity of the method—we take the sum of different code metrics (we multiply Readability and Maintainable Index by -1 to reverse their value). We provide the same weight to all the code metrics and the possible impact of this decision is discussed in the Threats to Validity (Section 4.4.3). Table 4.10 shows two sample bugs with the calculated code metrics, the normalized code metric, and also the Sum Complexity values.

Table 4.10: Two sample bugs from Severe and Non-Severe categories with their code metric values, normalized values, and the Sum Complexity values. The value in each row and the value of Sum Complexity are rounded to two precision.

Metrics	Severe		Non-Severe	
	Raw	Normalized	Raw	Normalized
LC	13.00	-0.35	532.00	21.27
PI	2.89	-0.25	8.61	2.75
MA	3.00	-0.37	194.00	23.50
NBD	1.00	-0.33	12.00	3.33
ML	2.00	-0.53	219.00	13.93
D	1.86	-0.39	18.84	2.20
FO	4.00	-0.35	40.00	22.14
E	220.14	-0.27	214131.00	45.50
MI	-103.93	-0.53	23.86	4.15
R	-0.22	-3.66	0	0
Sum Complexity	143.74	-7.07	215459.26	138.80

After calculating the SC value, to find the contradiction, we sorted our samples based on the SC (normalized version) value in ascending and descending orders separately and keep the first 60 samples in both cases for both datasets which result in 240 samples:

$$60 * 2 \text{ (Severe — Non-Severe)} * 2 \text{ (Defects4J — Bugs.jar)} = 240$$

For preventing bias, we randomly select 60 bugs out of these 240 bugs distributed evenly between Severe and Non-Severe bugs (our two categories in this RQ) and also between Defects4J and Bugs.jar (our datasets). Then, we provide the buggy code and the corresponding fixed code of these samples to the first and third authors of the second paper mentioned in the Contributions section.

We use a grounded theory approach called Card Sorting [85] to label the bug type manually. In this approach, we asked the first and third authors to label each buggy method with one or more bug types of their choice, based on their own-defined list of bugs types (e.g., Network, UI/GUI, IndexOutOfBounds, Math calculation), by looking at the buggy code and the fixed code only. The total unique labels created by the two authors in this phase were nine low-level labels (mentioned in the 4.2.3 section).

Then, the first and third authors had a meeting to discuss their assigned labels and also merge them into the final set of high-level labels containing four labels. During the merging process, while authors agreed on most of the samples' labels, several assigned labels had similar meanings but were expressed in different terms. We resolved those conflicts by assigning a more generic label that covers both original labels. For example, there is a sample where the first author assigned "I/O bug", as the label, but the third author assigned "Network bug". We merged these two labels in I/O bug label which covers Network bug too. Also, there are a few cases where the first and third authors cannot make the decision because of the conflicts, so we resolved these conflicts by comparing both provided categories and the code. Following this approach, we ended up with these four labels per bug: Integration, Edge/Boundary/ Security, and Specification.

Design of RQ3-2: Based on RQ2 results, we found that the SpotBugs tool missed or mislabeled bug severity in many cases, which may lead to undesirable consequences. For example, mislabeling the high-severity bugs with low severity values will reduce the bug's importance. Thus the development team may postpone the fixing process. Therefore, in this sub-RQ, we want to find out the reasons behind why these tools are not able to find the bugs or their severity values.

To do so, we first manually analyze 25 randomly chosen severe bugs (Blocker, Critical, Major, and High severity categories) in both datasets that are missed by static analysis tools, by comparing the buggy code and fixed code. In this step, we mainly want to know if the lack of bug patterns in static analysis tools prevents them from detecting these bugs, or if these bugs' nature is such that static analysis tools cannot detect them regardless of what pattern/rule they implement. We will provide a few samples in the following section including buggy and fixed code.

We then study a set of 25 randomly selected samples of severe bugs (Blocker, Critical, Major, and High severity categories) which are detected by SpotBugs tools, but the reported severity is lower than the actual severity values, e.g., when the actual severity value is Critical, but the tools report the Low severity value.

Design of RQ3-3: Given the results of RQ1 and RQ2, in this sub-RQ, we want to see if both severity estimation approaches (based on code complexity and static analysis tools) are leveraging similar or different characteristics of the code to predict severity or not. The motivation behind this sub-RQ is that if they are focusing on different aspects, combining both approaches will lead to a better prediction.

To answer this question, we manually investigate all the samples predicted by the SpotBugs to see if the calculated Sum Complexity (mentioned in the RQ3-1) values are aligned with the reported bug severity by this tool. We report samples where these values are contradictory, and one of them predicts the bug severity, accurately, in the following section. For instance, we look at a case where there is a high-severity buggy

method in the dataset, which has a large Sum Complexity value, but the SpotBugs reports Low severity. We also summarize cases where using both of these approaches has better performance than using either of them in the following section.

Results

RQ3-1 Answer: After applying the Card Sorting approach which is explained in Section 4.2.3 we found nine low-level bug labels such as UI (User Interface) bug, multi-threading bug, and math calculation bugs. These labels may have overlap in many scenarios, and some of them contain only one or two samples.

Table 4.11: Distribution of randomly sampled bugs with low-level bug labels.

Low-level Label	Number
I/O	2
UI	1
Multi-threading	2
Null checking	4
Exception handling	4
Math/String calculation	3
Wrong/missing objects and parameters	21
Input/state checking	16
Completely wrong implementation	7

Table 4.11 shows bug types and their distributions. **I/O** bugs occur in a method where the code access any I/O such as a file or network. **Multi-threading** bugs are due to not handling threading issues such as deadlock. **Null checking** bugs happen when the code access the null object or do not handle special specification when the method argument is null. **Exception handling** bugs occur when the method does not handle exceptions when a variable/object has a special value. **Math/String calculation** bugs occur when the mathematical calculations or string manipulations are performed wrongly. **Wrong/missing objects and parameters type** refers to code where the developer passes a wrong parameter to a method or use the wrong object to call. **Input/state checking** bugs happen when the code does not validate the function inputs or it does not consider the specific case of an object state for having different logic. **Completely wrong implementation** bug type shows cases where the fixed code is changed completely to implement new logic compared to the buggy code.

We provided the distributions of high-level bug labels in case of severe bugs and low Sum Complexity (SC), and also non-severe bugs and high Sum Complexity (SC) in Table 4.12. The number of Security faults is more when the bugs are labeled as severe, but the code metric values are low which indicates that security bugs may appear in simple methods but may lead to severe bugs. Furthermore, Edge/Boundary faults are

Table 4.12: Distribution of high-level bug labels where there is contradiction between severity values and Sum Complexity (SC) value.

High-level Label	Severe Bug & Low SC	Non-severe Bug & High SC
Integration	2	0
Edge/Boundary	2	7
Security	7	3
Specification	19	20

mostly available in samples with high Sum Complexity (SC) values where the code is complex by having many different branches and statements where developers tend to miss edge/boundary cases, but these bugs are not severe necessarily.

In the following, we will provide a definition for each of these four bug type categories and give some samples per category. Just a reminder that these categories have been defined based on our dataset and are the result of merging the labels from the Card Sorting phase.

Integration faults: This bug label exhibit cases where the bug is related to another component, module, or class that our buggy method is using, so it would be impossible to access the buggy method without considering the called module(s). For example, Fig 4.7 shows a buggy code and its corresponding fixed code in the `JXPath-14` project. The buggy method seems to be very simple containing only three statements, so based on the code metric values this is not a complex code, but the actual severity value is high. The bug is related to the `floor()` method which is from another module (`math`). It seems that this module may not handle `NaN` and `Infinite` values or it may handle them differently than the studied project’s (`JXPath`) specification. From the patch, we see that the developer needs to return a different value when the `v` parameter is `NaN` or `Infinite`, so the developer had to handle this case in the code before calling the `Math.floor()` function. As another example, the buggy method in the `Cli-2` project uses the `NumberUtils.createNumber()` method to convert the variable with `String` type to the `Number` type, but the patch shows that the developer has rewritten the logic of converting `String` to `Number` inside the code without using the `NumberUtils` anymore. In this case, the called component may contain a bug or it may be developed with different specification which does not satisfy the caller component’s developer’s requirement, so for detecting the severity value of this kind of bug accurately it is required to consider both caller and called module code together.

Since we found only two Integration faults, we do not draw a conclusion about the correlation between the Integration faults and their high severity. However, our observations show that estimating the severity of faults in this category by only considering the caller method seems impossible. Thus an analysis in a coarser

```

protected Object functionFloor(EvalContext context) {
    assertArgCount(1);
    double v = InfoSetUtil.doubleValue(getArg1().computeValue(context));
+   if (Double.isNaN(v) || Double.isInfinite(v)) {
+       return new Double(v);
+   }
    return new Double(Math.floor(v));
}

```

Figure 4.7: Sample bug with High severity value in the JXPath-14 project of the Defects4J dataset.

```

@Override
public double doubleValue() {
    double result = numerator.doubleValue() / denominator.doubleValue();
+   if (Double.isNaN(result)) {
+       // Numerator and/or denominator must be out of range:
+       // Calculate how far to shift them to put them in range.
+       int shift = Math.max(numerator.bitLength(),
+                           denominator.bitLength() - Double.MAX_EXPONENT;
+       result = numerator.shiftRight(shift).doubleValue() /
+               denominator.shiftRight(shift).doubleValue();
+   }
    return result;
}

```

Figure 4.8: Sample bug with High severity value in the Math-36 project of the Defects4J dataset.

granularity (e.g, class/module/package) is suggested.

Edge/Boundary cases: This kind of bug happens when the developers do not handle edge/boundary cases which usually leads to a software crash. Several samples in this category contain a severe bug with a simple code according to our code metrics, containing only a few statements. For example, Fig 4.8 shows the buggy method in the Math-36 project, where the developer did not handle the edge case (`Double.isNaN(result)`), but the patch shows that the studied method should have performed an extra calculation when the `result` variable is NaN. This bug may lead to a software crash, unauthorized access, or undesired behaviour.

Another severe bug exists in the `accumulo` project where the buggy method renames a requested table. In this method, the code does not check if the requested table exists already or whether the new requested

```

@Override
public boolean isGranted(@Nonnull String path, long permissions) {
-   Iterator<PermissionEntry> it = getEntryIterator(new EntryPredicate(
-       path, Permissions.respectParentPermissions(permissions)));
-   return hasPermissions(it, permissions, path);
+   EntryPredicate predicate = new EntryPredicate(
+       path, Permissions.respectParentPermissions(permissions));
+   return hasPermissions(getEntryIterator(predicate), predicate, permissions, path);
}

```

Figure 4.9: Sample bug with the Major severity value in the OAK-3324_5f863af6 project of Bugs.jar dataset.

name is already taken by another table, so this bug probably leads to a software crash or overwriting the existing table.

We found that most of the Edge/Boundary bugs are not severe in our datasets, but they appear in complex methods. Measuring the bug consequences in the whole system would be required to find the bug impact and eventually the severity. Edge/Boundary case testing could be a possible solution to determine the bug impact based on the number of failed tests.

Security faults: Security faults can be exploited to gain unauthorized access/privileges by intruders. Since it is not easy to find if there is a security fault, solely from the source code, we found that some bugs are highly potential to be exploited by intruders by considering keywords and the method context.

For example, there is a bug with Major severity in the `jackrabbit-oak` project which is shown in Fig 4.9. This simple (based on the code metric values) method checks whether the requested path argument has the access permission to be used. Handling permissions incorrectly may potentially lead to unauthorized access, and this usually has harmful consequences such as data loss or breaking the system completely.

After investigating all the samples in this category, we found that Security faults are severe (Blocker, Critical, Major, High) in most cases. However, these bugs are available in simple methods (based on our code metrics) that handle a sensitive task which may lead to harmful consequences and result in severe bugs.

Although we found most of our Security faults happen in the simple methods, but more specific methodologies such as security testing are required to find these bugs' severity values. If we can find the security faults type (e.g., Injection, Data Exposure, Broken Access Control, etc) the bug severity can be found easily, since security fault types typically have a specific ranking based on their probable consequences in a few defined standards.

Specification faults: This category, which is our largest category, exhibits bugs where developers have

```

    case Token.ASSIGN:
        if (parent.getFirstChild() == n) {
            isSet = true;
            type = getValueType(n.getNext());
        }
        break;
    case Token.GETPROP:
        return;
    case Token.FUNCTION:
        Node gramps = parent.getParent();
        if (gramps == null || NodeUtil.isFunctionExpression(parent)) {
            return;
        }
        isSet = true;
        type = Name.Type.FUNCTION;
        break;
+   case Token.CATCH:
    case Token.INC:
    case Token.DEC:
        isSet = true;
        type = Name.Type.OTHER;
        break;
    default:

```

Figure 4.10: Sample bug with the Low severity value in the Closure-119 project of the Defects4J dataset.

```

char ch = cs.charAt(i);
for (int j = 0; j < searchLength; j++) {
    if (searchChars[j] == ch) {
-         if (i < csLast && j < searchLast &&
-             ch >= Character.MIN_HIGH_SURROGATE &&
-             ch <= Character.MAX_HIGH_SURROGATE) {
-             // missing low surrogate, fine, like String.indexOf(String)
-             if (searchChars[j + 1] == cs.charAt(i + 1)) {
+                 if (Character.isHighSurrogate(ch)) {
+                     if (j == searchLast) {
+                         // missing low surrogate, fine, like String.indexOf(String)
+                         return true;
+                     }
+                     if (i < csLast && searchChars[j + 1] == cs.charAt(i + 1)) {
+                         return true;
+                     }
+                 } else {
+                     // ch is in the Basic Multilingual Plane
+                     return true;
+                 }
+             }
        }
    }
}

```

Figure 4.11: Sample bug with the Low severity value in the Lang-30 project of the Defects4J dataset.

not implemented the full specification (e.g., the logic related to a partition in the input space has been ignored, i.e., misrepresented). This bug type is called Specification faults since they may not occur if the developers would follow the specification of the software completely [51]. Patches for these bugs contain changing conditional statements, math calculations, string manipulation, or a complete rewrite of the code.

For example, there is a bug with the Low severity value in **Closure-119** project where the buggy method is too complex (containing 120 lines of code having many nested **switch-case** and conditional statements), but the patch is very simple which handles a small part of input space. Part of this method and its patch are shown in Fig 4.10. Long methods such as our mentioned example are usually bug-prone, and they are detected as buggy by code metrics correctly, but the bug could have a Low severity value since these long and complicated methods may handle a not sensitive task such as handling the GUI part.

Also, Fig 4.11 shows part of a buggy method from the **Lang30** project which has a Low severity value. In this sample, the patch is not as simple as the previous example and part of the code is rewritten, but the severity is still low.

The number of Specification faults in both severe and non-severe categories is almost equal as shown in

Table 4.12, so further information is strongly required to detect the bug severity in this case.

Summary of RQ3.1 Results: Most of our studied bugs were placed in the Specification categories, evenly, regardless of their severity value. Our studied bugs in the Edge/Boundary category are complex (high SC value) but mainly have a Low severity value. Security bugs have High severity, but they are mostly not complex (low SC value). It is almost impossible to detect the Integration fault severity label without considering the called module/class.

RQ3-2 Answer: In this section, we provide the results of our manual investigation of severe bugs (Blocker, Critical, Major, High) that are not detected by static analysis tools and go through some examples of severe bugs that are miss-labeled by SpotBugs.

Fig 4.12 shows a Critical bug in **Cli-2** project, which is not detected by SpotBugs and Infer because it cannot be fitted into any rule. The provided patch seems to be simple, but the bug is too complex, since it is related to a specific case where the developer should use `token` variable and also the `break` statement in the last `else` branch to handle a special case required by the specification.

Another example is a Critical bug in the **Math-11** project which is shown in Fig 4.13. The patch is very simple, and the buggy statement looks almost identical to the fixed statement such that it is even very hard for a human to find the issue without knowing the special values of `dim` variable that leads to the bug.

We found other severe bugs that cannot be generalized in the bug patterns such as a bug in the **Math-105** project where the `getSumSquaredErrors()` in the buggy code returns the maximum number between the error value and zero wrongly. The patch shows that it should return the calculated error value only (there is no need for the max function). Also, **Codec-18** project contains a bug where the `equals` method check if two provided `CharSequence` objects are equal. The patch shows that the developer implemented the equality condition wrongly by not considering the maximum length of these strings which are patched in the fixed version.

In all of the mentioned examples, we found that these bugs are so complex that cannot be mapped to any of the predefined patterns. Detecting these severe bugs by only leveraging the source code (even at the coarse-level granularity such as class/package/module) is almost impossible. Therefore, more sophisticated patterns need to be implemented by static analysis tools. One may also try using dynamic approaches such as testing to detect these types of bugs.

Furthermore, we found some samples when there is a specific rule for the bug in the tools, but the bug is not detected by any of the tools. For example, the buggy method in the **Cli-9** project copies the values of a `mapping` variable (defined at the class level) to the provided argument of the method. The provided patch

```

protected void burstToken(String token, boolean stopAtNonOption)
{
    int tokenLength = token.length();

    for (int i = 1; i < tokenLength; i++)
    {
        String ch = String.valueOf(token.charAt(i));
        boolean hasOption = options.hasOption(ch);

        if (hasOption)
        {
            tokens.add("-" + ch);
            currentOption = options.getOption(ch);

            if (currentOption.hasArg() && (token.length() != (i + 1)))
            {
                tokens.add(token.substring(i + 1));

                break;
            }
        }
        else if (stopAtNonOption)
        {
            process(token.substring(i));
        }
        else
        {
            -           tokens.add("-" + ch);
            +           tokens.add(token);
            +           break;
        }
    }
}

```

Figure 4.12: Critical bug in the Cli-2 project of the Defects4J dataset.

```

public double density(final double[] vals) throws DimensionMismatchException {
    final int dim = getDimension();
    if (vals.length != dim) {
        throw new DimensionMismatchException(vals.length, dim);
    }

    - return FastMath.pow(2 * FastMath.PI, -dim / 2) *
    + return FastMath.pow(2 * FastMath.PI, -0.5 * dim) *
        FastMath.pow(covarianceMatrixDeterminant, -0.5) *
        getExponentTerm(vals);
}

```

Figure 4.13: Critical bug in the Math-11 project of the Defects4J dataset.

shows that there is a specific condition when the `mapping` variable is `null`, so the buggy method should return the provided argument instead of copying data. Although the SpotBugs tool has some predefined rules regarding the `null` handling of variables in the method it cannot report this method as buggy.

One possibility for this low effectiveness by static analysis tools is that since these tools are heavily used in production as a part of the development environment or building process, there is a high chance that all of these projects were scanned by these tools after each commit/release, so all bugs that can be detected by these tools have been fixed already.

By investigating the results of the SpotBugs tool to see why this tool mislabeled the severity of many bugs we found that 70 bugs are detected to have lower severity values than their actual severity value. After analyzing these samples, we found two scenarios. The first one would be samples that SpotBugs reported another bug in the buggy method than the actual bug. In the second case, Spotbugs reports the bug correctly, but it assigns the lower severity labels.

For example, the reported bug in the **Math-91** project has High severity, but SpotBugs reports Low severity. The actual reported bug is due to the equality problem of two **Fraction** objects because of the limited precision ³, but the SpotBugs reports the `CO_COMPARETO_INCORRECT_FLOATING` as the issue type which is in the `BAD_PRACTICE` category with the following description:

“This method compares double or float values using pattern like this: `val1 > val2 ? 1 : val1 < val2 ? -1 : 0`”

Although this statement exists in the buggy method, the actual reported bug is not happening because of this statement.

Another example is the bug in the **Compress-12** project where the actual reported bug is related to throw-

³<https://issues.apache.org/jira/browse/MATH-252>

ing the `IllegalArgumentException` instead of `IOException` on corrupted files ⁴. However, the SpotBugs reported this method containing a Low-severity bug with the `I18N` issue type in the `DM_DEFAULT_ENCODING` category with the following description:

“Found a call to a method which will perform a byte to String (or String to byte) conversion, and will assume that the default platform encoding is suitable.”

In the mentioned examples we found that SpotBugs reported a different bug with Low severity instead of the actual bug with High severity.

Conversely, there is a Major bug in the `accumulo-209_397f86f6` project regarding the multi-byte character encoding ⁵, and the SpotBugs reported a `DM_DEFAULT_ENCODING` as the issue type in `I18N` category with the following description:

“Found a call to a method which will perform a byte to String (or String to byte) conversion, and will assume that the default platform encoding is suitable.”

It seems that the SpotBugs has detected the bug correctly, but it assigns a low severity to it. Based on our best knowledge, SpotBugs assign the severity of the reported bugs based on their implemented patterns, so it always reports Low severity for this kind of bug. However, in practice, one specific issue may lead to a severe or non-severe bug in different cases such as the number of affected users or the project timeline.

Summary of RQ3.2 Results: SpotBugs missed most of the studied bugs regardless of their severity value because of no matching rule for those bugs. In specific cases there are matching (almost the same) rules, however, the bugs cannot be detected accurately. Bugs are assigned different (lower) severity by SpotBugs since it detected the severity value based on the general pre-defined bug patterns, but in practice, a specific bug may have different severity values in different projects based on the number of affected users or the project timeline.

RQ3-3 Answer: By comparing the calculated SC (mentioned in 4.2.3) and the severity predicted by the SpotBugs tool we found some samples where using both of these results will be more useful.

For example, we consider the buggy method `computeShiftIncrement()` in the `Math-87` project has High severity value. The SpotBugs tool reports the Low severity value, but the SC value shows that this method is a complex method regarding the code metrics we used. In this case, the code metric values exhibit the severity better.

Conversely, there is a buggy method named `parse()` in the `Closure-68` project with the Medium severity

⁴<https://issues.apache.org/jira/browse/COMPRESS-178>

⁵https://github.com/bugs-dot-jar/accumulo/tree/bugs-dot-jar_ACCUMULO-209_397f86f6

value. The SC value for this buggy method is very large which shows the complexity of the method, so we may assume this is a severe bug, but the SpotBugs reports the Low severity value. In the mentioned example, the SpotBugs prediction is closer to the actual severity value than the code metric values.

The above-mentioned examples show that these two approaches focus on different aspects of bugs, so considering both of them can lead to better prediction than using only one of them. These two approaches may not have the same weights in the hybrid approach, so determining the correct weights for each of the code metric approaches and the static analysis tools approach should be investigated and determined. From our observations, static analysis tools have very low performance, so considering a large weight for this approach may lead to poor results in the end. However, the weights may vary in different projects based on the project properties.

Summary of RQ3.3 Results: Considering both code metrics and static analysis tools together may lead to promising performance since they leverage different characteristics of bugs in some cases. Since static analysis tools can only find relatively small numbers of bugs, so considering a large weight for them in the hybrid approach may lead to incorrect detection.

4.2.4 Representation Learning & Bug Severity (RQ4)

The objective of RQ4 is to find out whether the representation provided by the pre-trained models (CodeBERT in our study) can help predict the bugs' severity, better than existing alternatives. Unlike, our previous experiments we do not perform the bug detection application using CodeBERT since it has been done already in the previous research named CodeXGLUE [74]. Researchers show that the CodeBERT model is able to perform the bug detection (defect detection) task as well as the state-of-the-art approaches, so we only consider the bug severity prediction task using CodeBERT which has not been explored yet.

Motivation

Since our results from the previous research questions show that code metrics and static analysis tools are not powerful enough to predict the bug severity, we aim to find if leveraging transfer learning by using code representation from large language models, such as the CodeBERT model, shows promising results.

The motivation behind this idea is that large pre-trained language models trained on different programming languages have recently shown outstanding results in many software engineering tasks including bug detection, clone detection, and program repair. Also, these models can be used for several programming languages so this technique can be easily applied to other datasets containing other languages supported by the models. In addition, our selected pre-trained model, CodeBERT, supports the natural language input,

so additional input such as bug descriptions provided by the developer or QA team can be used in addition to the code which may improve the results. Furthermore, since these models have been trained on a very large dataset they can be used in studies or practical projects where there is a small dataset available by fine-tuning the model and leveraging the already trained features by the model itself.

Approach

We pose two sub-RQs to find the effectiveness of representation learning in predicting bug severity labels.

RQ4: Can code representation provided by the CodeBERT model predict the bug severity?

RQ4-1: Can the CodeBERT model predict bug severity from the source code?

RQ4-2: Does adding source code metrics improve CodeBERT effectiveness?

Design of RQ4-1: We selected the “codebert-base” model from Hugging Face [5] which is initialized with “roberta-base” and trained on bi-modal data (documents & code) with MLM (masked language modeling) and RTD(replaced token detection) objective.

Since our selected model has been trained on the large dataset so there is no need to study its effectiveness per project, so we merge all of our samples from the Defects4J and Bugs.jar dataset to make a uniform dataset and change the USL labels to the numerical to be applicable as our model target variable which is shown the Table 4.13.

Table 4.13: Mapping USL types to the numerical format for being used as a target variable in the training process.

Unified Severity Label (USL)	Severity Target Variable
Critical, Blocker	0
Major, High	1
Medium	2
Low, Trivial, Minor	3

Also, we have removed any comment (JavaDoc or inline format) before or in the middle of the method source code because we do not want to consider the code comments effect in this study, but we keep the JavaDoc comments as a separate feature in our dataset which can be used in the future works.

Furthermore, to remove the potential scaling problem of the numerical data, we use the RobustScaler[8] (mentioned in Section 4.2.3) to scale the source code metrics features.

We remove potential duplicated samples from our dataset, and then shuffle and split it into three train,

validation, and test splits using 70%,15%,15% ratio. We did not perform cross-validation technique to keep our study align with previous studied research such as CodeXGLUE [74] and the CodeBERT [38] provided experiments. However, running cross-validation techniques such as K-Fold may affect our obtained results which can be investigated in future work.

To the best of our knowledge, there is no study considering source code metrics or code representation for predicting bug severity, we use several classic machine learning models as a baseline since they have been used for bug detection (using code metrics) or severity predictions(using bug report) in the previous literature. We train KNN, SVM, NaiveBayes, DecisionTree, RandomForest, XgBoost, and AdaBoost models by using the only 10 source code metrics we have calculated in the RQ1. Since the hyperparameter tuning is out of the scope of our study we keep the default hyperparameters for these models available in the Scikit-learn library [26].

We report several evaluation metrics such as accuracy, F1 score (weighted), precision, recall, and ROC-AUC score, but we consider the F1 score for comparing the models since it considers both precision and recall and it works well in case of having an imbalanced dataset.

We add `RobertaClassificationHead` layer to the CodeBERT model to follow the architecture named `RobertaForSequenceClassification` exactly. We re-implemented this architecture ourselves instead of using `RobertaForSequenceClassification` directly since we require enough flexibility to leverage the output since it will be the base model for the following sections. This architecture uses the Softmax layer since our classification problem is a multi-class problem and also it uses the CrossEntropyLoss for calculating the loss.

We leverage this architecture for predicting the severity label, and since our design does not require any natural language input, we just provide buggy code to the model and the severity label as the target value.

We first freeze the CodeBERT model layers to prevent changing the weights of the existing layers drastically and only train our added classification layer by a large number of epochs (40 epochs). During the training step in each epoch, we use the early stopping technique with `patience=3` to prevent the model over-fitting problem by considering the loss value of the model using the validation set. After that, we unfreeze the CodeBERT model layers and fine-tune the whole architecture by using a smaller learning rate and fewer epoch numbers to adjust all layers' weight to follow our downstream tasks. In the fine-tuning step, we use 5 epochs since it is suggested in previous research [35] to use fewer epoch numbers to prevent the over-fitting problem and not changing the pre-trained model weights drastically. During the training process, we use the train split for training the model and the validation split for evaluating the model efficacy, and we save the checkpoint of the model with the best F1-weighted score. Finally, we use the test split which is never seen during the training process for the final evaluation of the model.

We use the 512 block size which is the maximum value supported by the CodeBERT, so whenever our

sample is shorter than this length we padded them with the `padding` token, and whenever the sample is larger than this length we truncate them by keeping the first 512 tokens.

Results

RQ4-1 Answer: For comparing the results of our comparison, we first provide the results of baseline models. Fig 4.14 and Fig 4.15 displays the ROC curve and Precision-Recall curve of baseline models respectively. According to these plots, the Random Forest model has the best performance in predicting the bug severity since for all different classes (severity labels) the plots are closer to the perfect classifier than other models.

Table 4.14 shows our results of these models with different evaluation metrics. As it is shown in the table the Random Forest model has the best performance by having better results regarding all of the provided metrics. SVM model has the weakest performance in predicting the severity values as its MCC value suggests that this model works like the no-skill (random) classifier. The F1-Perclass metric shows that this model only predicts the “class 1 (Major, High) severity” correctly and its low precision value means that it returns a lot of false positives.

As we do not perform any hyper-parameter tuning, our results may not reflect the models’ best performance for this classification task. Since we do not perform any hyper-parameter tuning for any models in this study, our comparison between different models in different RQs is not biased because of this decision. However, we discuss the potential threats regarding this decision in Section 4.4.

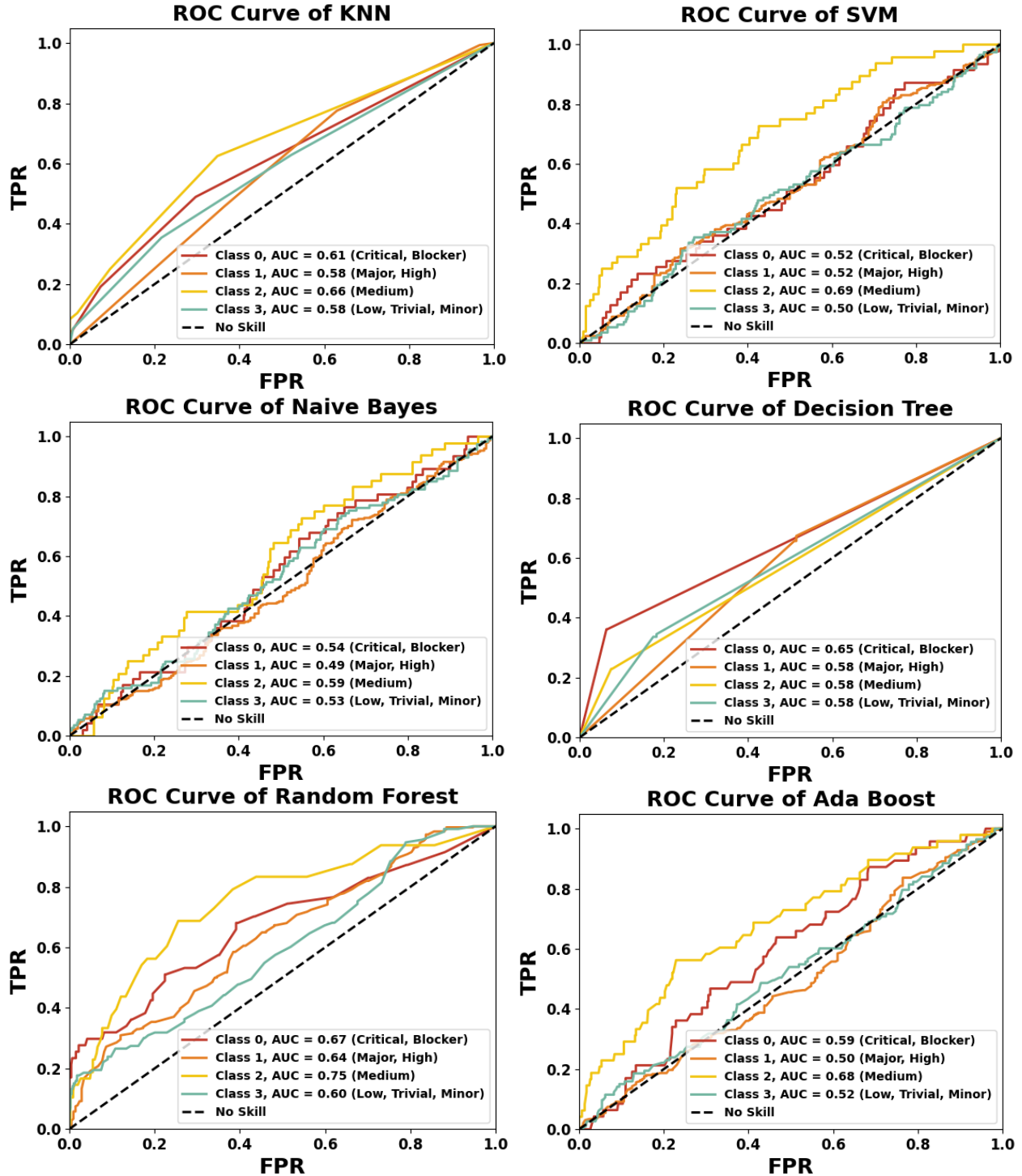
The confusion matrix of all baseline models is provided in Fig 4.16. The figure shows that the SVM model predicts the “class 1” label for every bug which confirms our result of Table 4.14 that this model acts like a random classifier. Also, from the MCC values provided in Table 4.14, we anticipate that the Random Forest and Decision Tree models to have the best results, and the confusion matrix confirms this since these two models have the highest numbers in their main diagonal. Random Forest models have better prediction capability in finding “class 1” labels since it detects 263 cases correctly while the Decision Tree detects 196 samples. Conversely, the decision tree models have a better capability of finding all other classes.

In addition, the Random Forest model works better in case of not predicting lower severity labels for severe bugs. For example, the Random Forest model does not classify any bugs of “class 0 (Critical, Blocker)” to be “class 2 (Medium)” or “class 3 (Low, Trivial, Minor)”, but the Decision Tree model predicts 12 samples of “class 0” to have “class 3” wrongly. This is not an expected behavior since the model underestimates the bug severity and may lead the technical team to ignore the severe bugs.

The performance of CodeBERT model is provided in Fig 4.17. Also, the evaluation metric values are listed in the Table 4.15. This model predicts the “Class 2 (Medium)” severity label well since the ROC and Precision-Recall curve of this class is very close to the perfect classifier curve. This result seems interesting

since the number of instances in our dataset with the “Class2 (Medium)” label is relatively low (489 in training split), but the model acts like a perfect classifier in predicting this severity label.

By comparing the results of this approach (Table 4.15) and the baseline models (Table 4.14), we can state that the CodeBERT model has significantly better efficacy in predicting bug severity. This model increases



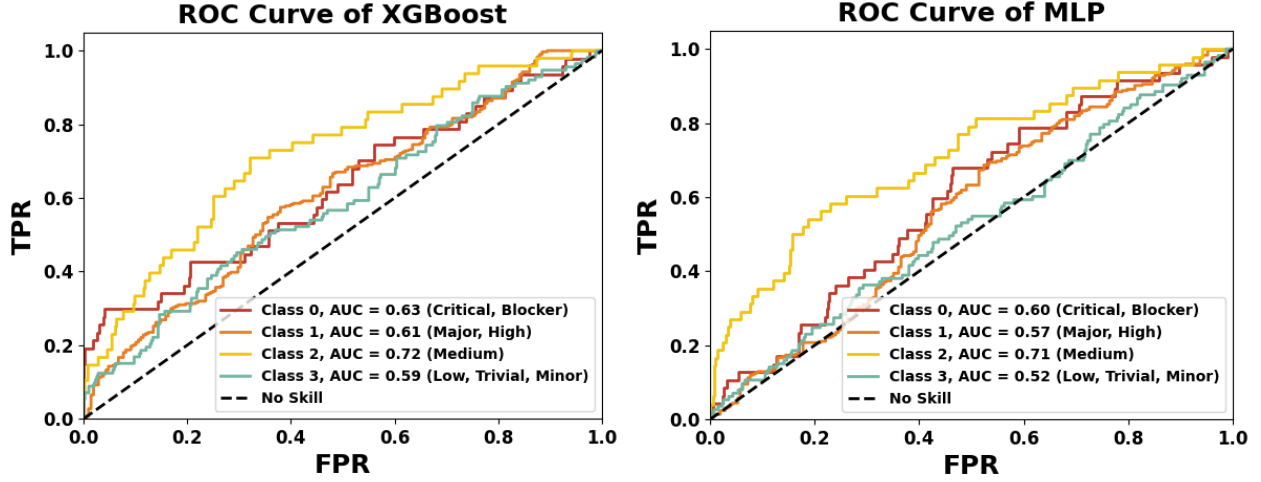


Figure 4.14: ROC curve of baseline models.

Table 4.14: Baseline models performance with different evaluation metrics. The bold line shows the model with the best performance.

Model	Precision	Recall	F1-Weighted	F1-Perclass	AUC	MCC
KNN	0.49	0.56	0.49	[0.16, 0.71, 0.15, 0.19]	0.60	0.11
SVM	0.34	0.58	0.43	[0.00, 0.73, 0.00, 0.00]	0.55	0.00
Naive Bayes	0.45	0.53	0.43	[0.05, 0.71, 0.05, 0.05]	0.52	0.02
Decision Tree	0.51	0.52	0.51	[0.36, 0.65, 0.23, 0.34]	0.59	0.17
Random Forest	0.59	0.60	0.54	[0.35, 0.73, 0.20, 0.26]	0.65	0.21
Ada Boost	0.43	0.56	0.44	[0.00, 0.72, 0.07, 0.06]	0.56	0.01
XGBoost	0.52	0.57	0.51	[0.29, 0.71, 0.22, 0.20]	0.63	0.14
MLP	0.47	0.59	0.45	[0.00, 0.74, 0.10, 0.04]	0.59	0.10

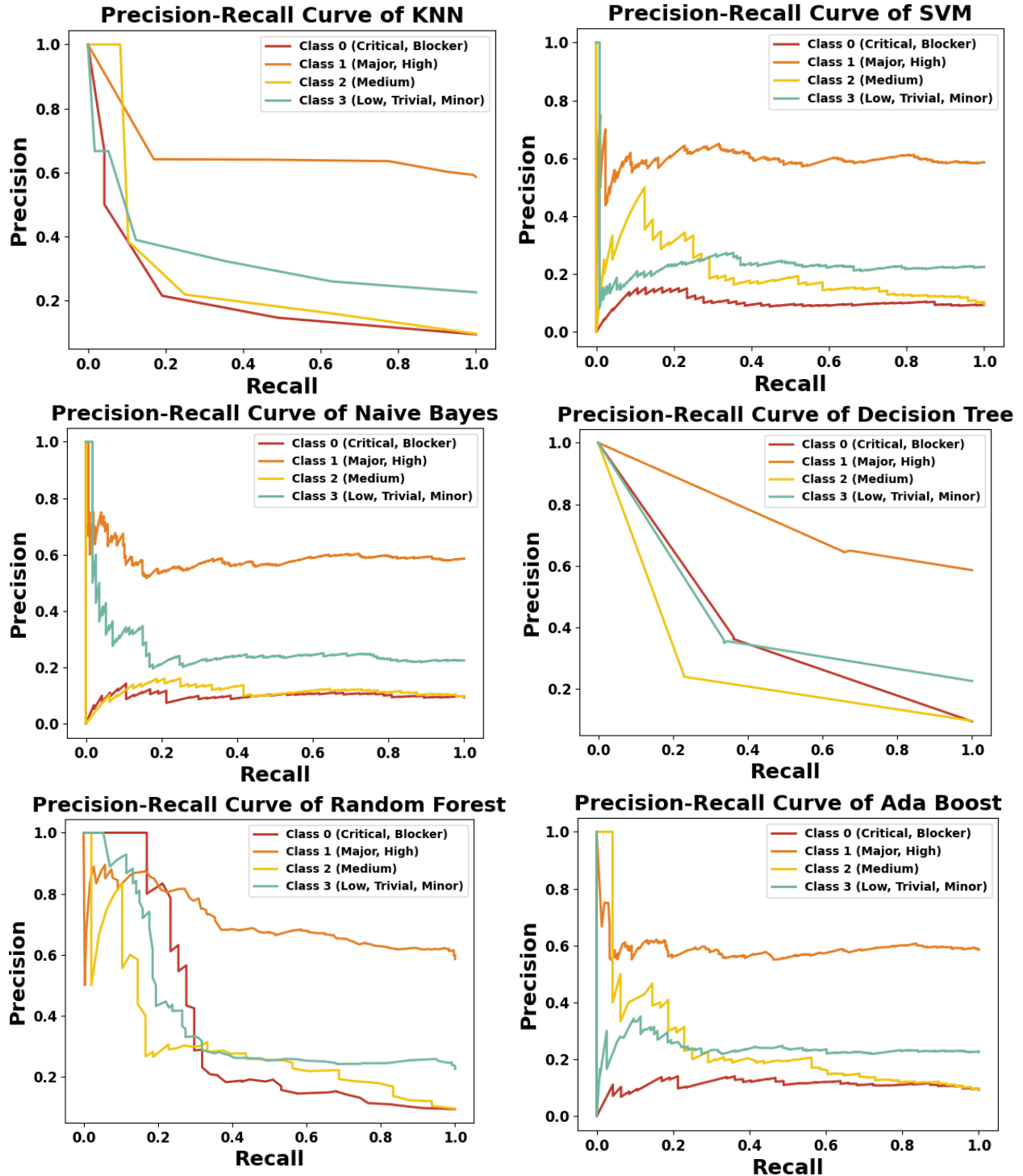
the F1-Weighted, AUC, and MCC values by 0.16 (29%), 0.22 (33%), and 0.30 (140%) respectively compared to the Random Forest model which shows the best results among our baseline models.

Table 4.15: CodeBERT model performance with different evaluation metrics.

Model	Precision	Recall	F1-Weighted	F1-Perclass	AUC	MCC
CodeBERT	0.71	0.72	0.70	[0.63, 0.79, 0.90, 0.42]	0.87	0.51

Furthermore, we have provided the confusion matrix of the CodeBERT model in Fig 4.18 where the actual values and the model prediction are shown for each class label. This figure shows that CodeBERT mislabeled 18 bugs with “class 0” severity; it predicts 15 of them as the “class 1” severity which means that it detected critical bugs as major severity bugs, but it only assign lower severity (medium, low) to 3 bugs. For the bugs with the “class 1” label, it predicted the severity in 87% cases. It classified all of the bugs with

the “class 2” label except for 6 bugs which it assigned higher severity which is not a harmful case. This miss-classification may increase the resource usage since developers or QA team members should consider these 6 bugs, but it does not increase the risk of using this model as opposed to situations where the model assigns the lower severity labels to the actual bugs (considering critical bugs as low-severity bugs). The bugs



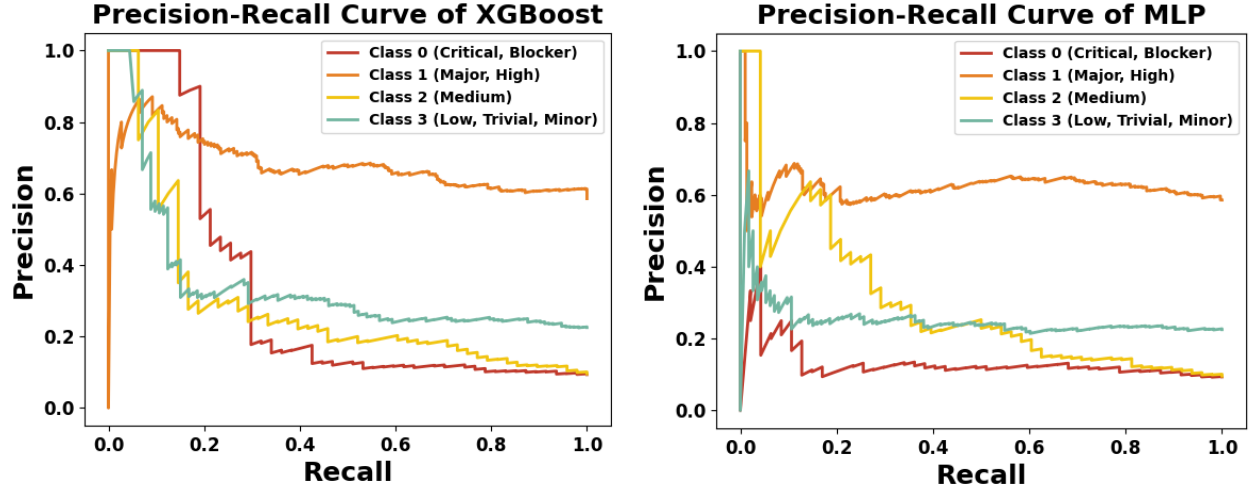


Figure 4.15: Precision-Recall curve of baseline models.

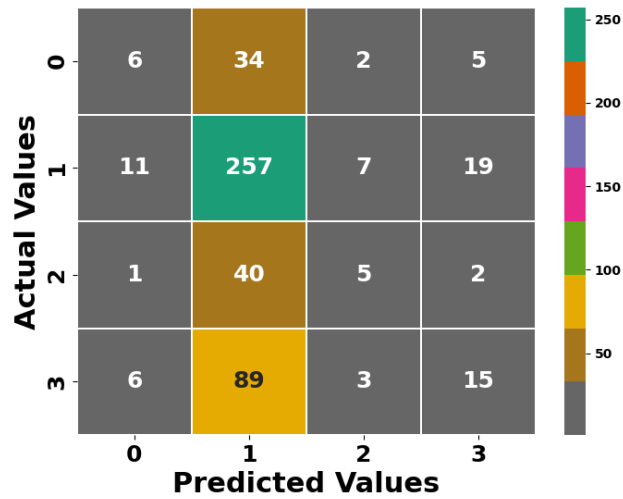
with the “class 3” labels are predicted to have higher severity in many cases. Since this miss-classification is due to assigning higher severity labels, we do not consider it a non-promising result. However, further investigation may reveal the reason behind this intention to assign higher severity labels, or improving the model design may reduce the number of miss-classification.

The results suggest that code representation has better capability in this downstream task compared to using the code metrics solely. The first potential reason is that in cases where the dataset is small, training a model from scratch may not lead to good results, but using pre-trained models and fine-tuning for the downstream task work well. The second potential reason is that the code metrics are not good indicators for predicting the severity solely, but the semantics behind the source code embedding and the relation between vectors of different source code embedding vectors can be used for such complex tasks as severity prediction.

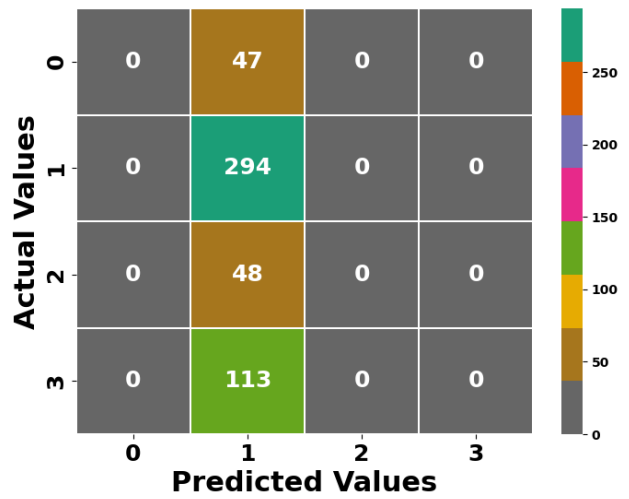
Summary of RQ4-1 Results: The Random Forest model shows the best performance among the baseline models. CodeBERT model has significantly increased the efficacy of the severity prediction task compared to the Random Forest by 29%, 33%, 140% according to the F1-Weighted, AUC, and MCC metrics.

Design of RQ4-2: To find the impact of our 10 code metrics we provide two different architectures for integrating our numerical code metric features with the pre-trained CodeBERT models. For the first architecture, we convert the code metric features to the textual format and provide it along with the source code to the model. Since the CodeBERT model has been trained on the textual data and considers the context around each word, it would be better to provide a context for each numerical feature. In this way, instead of providing code metrics values directly, we provide a paragraph describing each code metric (context for the value) and also put the metric values. The paragraph describing the code metrics with related code

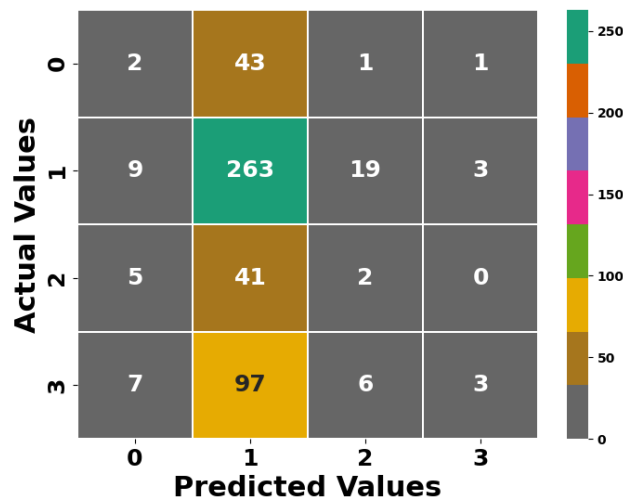
Confusion Matrix of KNN



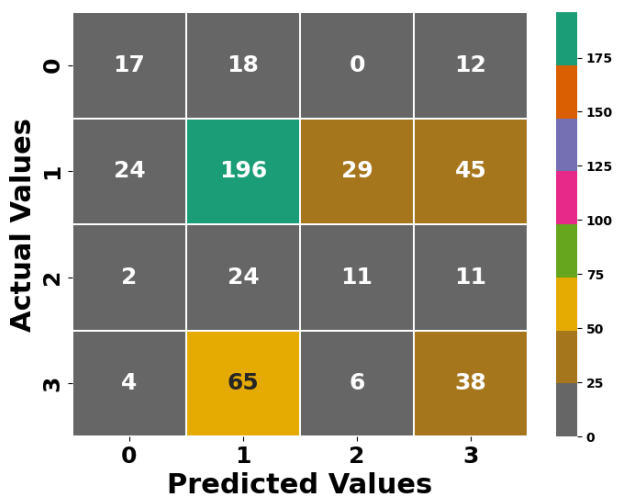
Confusion Matrix of SVM



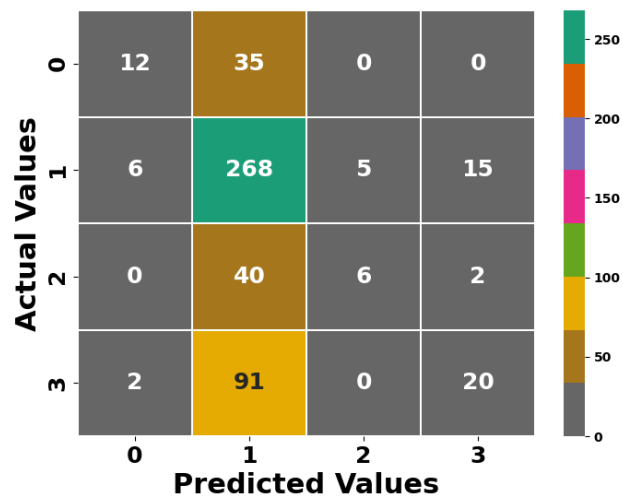
Confusion Matrix of Naive Bayes



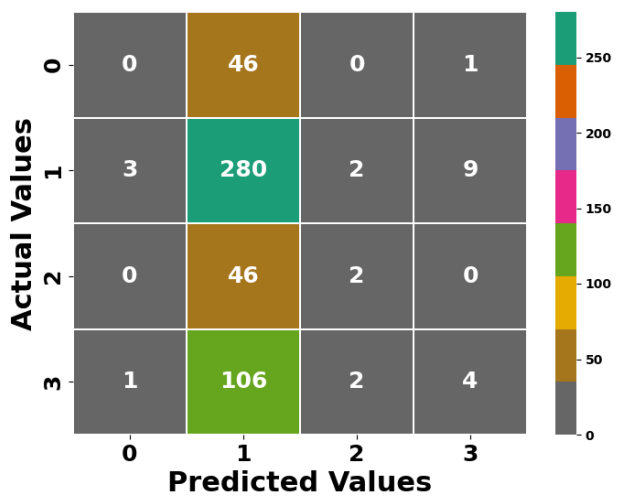
Confusion Matrix of Decision Tree



Confusion Matrix of Random Forest



Confusion Matrix of Ada Boost



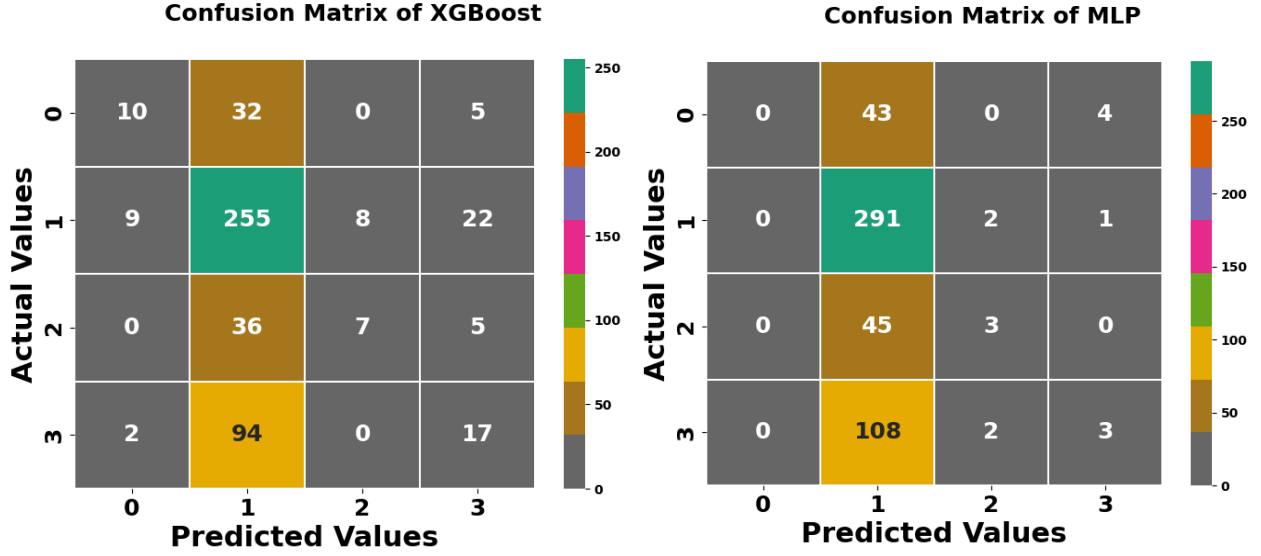


Figure 4.16: Confusion matrix of baseline models.

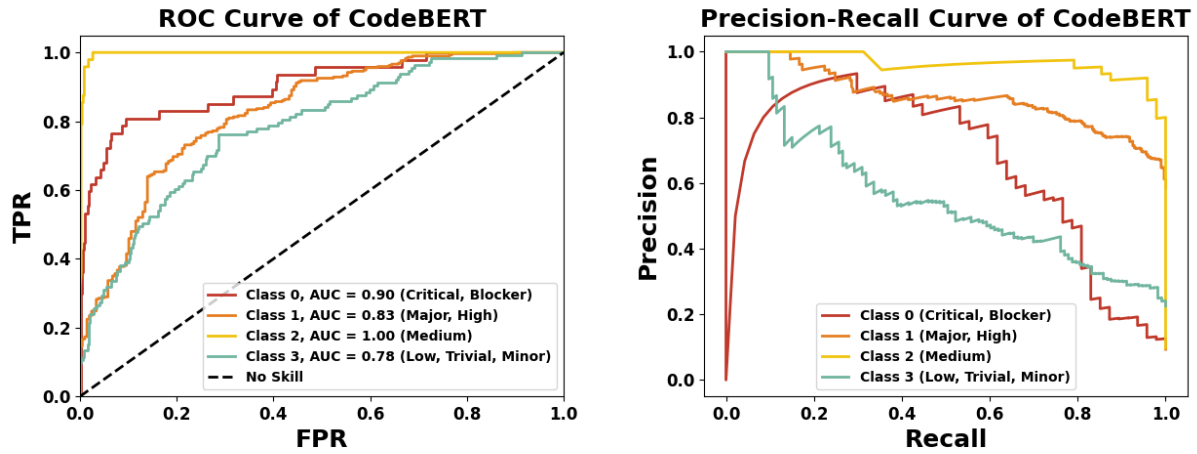


Figure 4.17: ROC curve and Precision-Recall curve of CodeBERT model.

metric values is shown in Table 4.16.

After creating this paragraph we put this paragraph in the first section of the whole input before the SEP token and then provide the source code input as shown below:

[CLS], Paragraph describing code metrics and their values ,[SEP], Source Code,[EOS]

In this architecture, our input contains two sections of NL(Natural Language) and PL(Programming Language), but we keep the whole section of NL and only truncate the PL section since the NL section (code metric values with their context) is short and constant for all samples but the source code section length

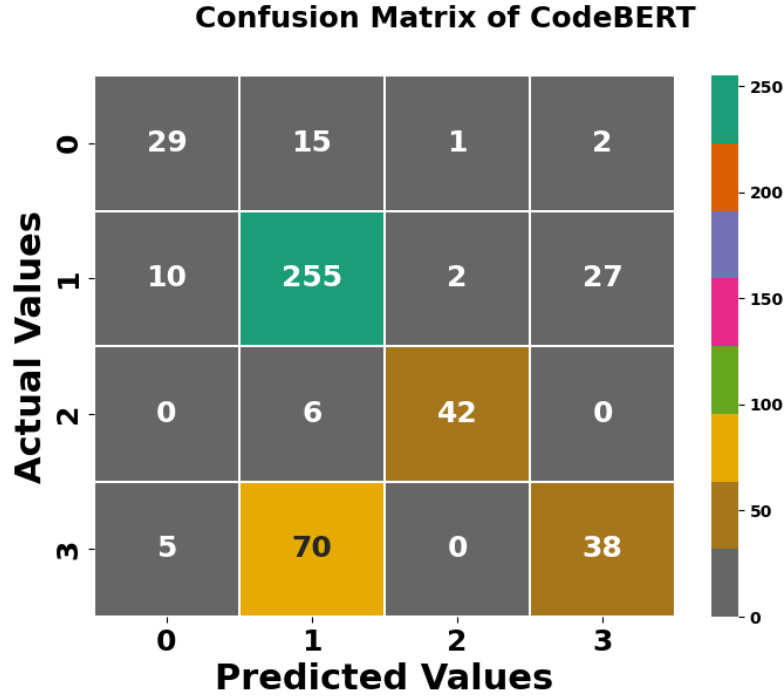


Figure 4.18: Confusion matrix of CodeBERT model.

Table 4.16: Creating context for code metric features to provide as the NL part of the model.

Code Metric	Value	NL Part of Model Input
LC	1	The code contains 1 lines and its complexity metrics values are 2, 3 and 4. The nested block depth is 5, and the difficulty of this code is 6. Maintainability score is 7 and this method calls 8 number of methods while its readability and effort metrics values are 9, 10
PI	2	
MA	3	
NBD	4	
ML	5	
D	6	
MI	7	
FO	8	
R	9	
E	10	

varies in different samples. Therefore, we truncate the PL part from the end to make the whole input size of 512 tokens. In addition, if the length of the final input is smaller than 512 we pad the input to the end with the **Pad** token.

The second architecture leverages the numerical features separately by concatenating the CLS token output with the vector containing all numerical features and then providing this vector to the dense layer. CLS token of the CodeBERT represents the source code input and our vector represents our numerical code metric features. By having this architecture, during the backpropagation phase, the CodeBERT model

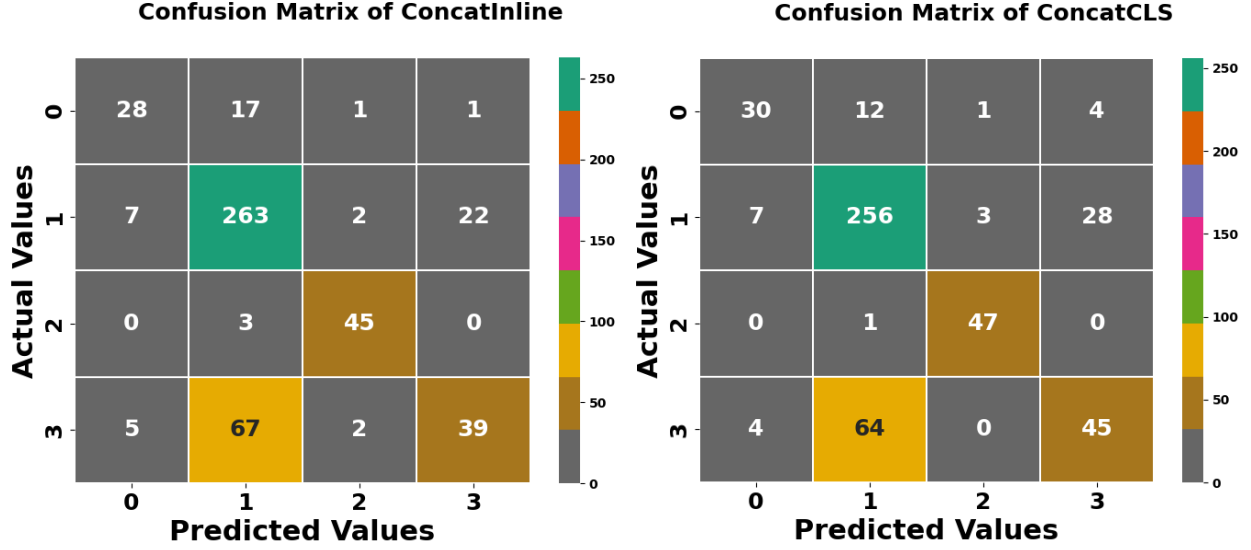


Figure 4.19: Confusion matrix of ConcatInline and ConcatCLS models.

weights are also updated according to the code metric features contributions.

RQ4-2 Answer: The ROC curve and Precision-Recall curve of our experiment using the ConcatInline model is shown in Fig 4.20. Also, the evaluation metrics are provided in Table 4.17. Our results show that adding source code metrics in the ConcatInline architecture has increased the performance of the CodeBERT model by 0.03, 0.01, and 0.04 for F1-Weighted, AUC, and MCC metrics respectively. In addition, the ROC curve shows that adding these metrics has increased the capability of the model to predict the “class 0 (Critical, Blocker)” severity label by 0.02 which shows the decisive role of the code metrics for high severity bugs.

ConcatCLS architecture has increased the performance of the CodeBERT model by 0.04, and 0.05 for F1-Weighted and MCC metrics respectively, but the AUC value is not improved. Also, from the F1-Perclass row, we find that the results are improved in all cases. Fig 4.19 shows the confusion matrix of both ConcatInline and ConcatCLS models. By comparing these figures with the confusion matrix of CodeBERT (Fig 4.15) we find that the ConcatInline model works better than the CodeBERT model for all classes except for the “class 0” label where the number of correct classification is reduced by 1. The ConcatCLS model has improved the number of correct classifications in all classes compared to the CodeBERT model.

The interesting result from these figures is that the ConcatCLS model decreased the number of miss-classification of “class 3” to “class 2” discussed in the RQ4-1 results (Section 4.2.4) from 70 to 64 (8.5%). Also, this model predicts the “class 2” labels in 98% cases.

For comparing the ConcatCLS and ConcatInline model we consider the confusion matrix and MCC

metrics since the confusion matrix provides better information about each label separately and MCC is a good metric in case of imbalanced datasets. Based on the discussion and figures we can state that ConcatCLS architecture outperforms the ConcatInline model.

By comparing the results of the ConcatInline, ConcatCLS, and CodeBERT models we see that the ConcatInline and ConcatCLS do not increase the efficacy of the CodeBERT model significantly, but the CodeBERT model increased the baseline models performance drastically. One of the reasons behind this little improvement would be the power of the CodeBERT model in finding the representation of the input such that CLS token representation understand the whole input better than our code metrics vectors. It seems that the semantic relationship between the tokens and their context leads to better severity prediction than the calculated code metrics which are extracted statically from the source code without considering any semantics. Also, this shows the power of pre-trained models since the CodeBERT has been pre-trained on millions of samples, so it can provide enough information about new inputs for required downstream tasks. However, by having many training samples, it may be possible to train a more complex MLP (Multi-Layer Perceptron) alongside the CodeBERT which may improve the overall performance.

Table 4.17: ConcatInline and ConcatCLS models performance with different evaluation metrics.

Model	Precision	Recall	F1-Weighted	F1-Perclass	AUC	MCC
ConcatInline	0.73	0.75	0.73	[0.64, 0.81, 0.91, 0.44]	0.88	0.55
ConcatCLS	0.73	0.75	0.74	[0.68, 0.82, 0.95, 0.47]	0.87	0.56

Summary of RQ4-2 Results: The ConcatInline and ConcatCLS models outperform the CodeBERT models by improving evaluation metrics. ConcatCLS model outperforms the ConcatInline model since it has a better MCC value and it decreased the number of miss-classifications cases where the CodeBERT and ConcatInline models predict lower severity labels for bugs.

4.3 Discussion

Our results from RQ1 indicate that all of the studied code metrics have significant performance in finding the buggy methods. However, the Line of Code, Maintainable Index, FanOut, and Effort metrics have the best performance which implies the importance of these metrics to be used in the research community and also in practice to warn developers about their code complexity. However, we found no significant relationship between studied code metrics and the bug severity which means high-severity bugs may exist in the simple code or low-severity bugs may exist in the complex code. Surprisingly, the Line of Code has is the worst

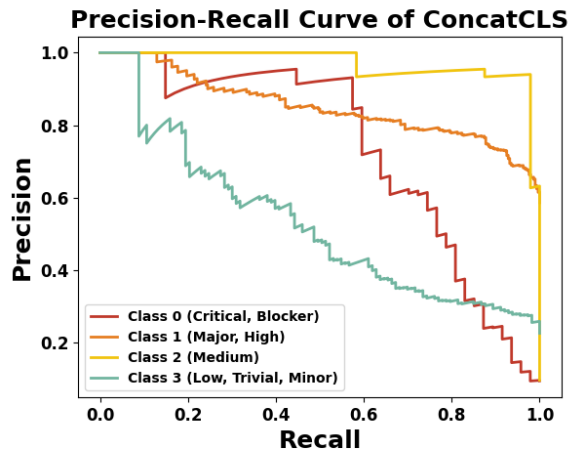
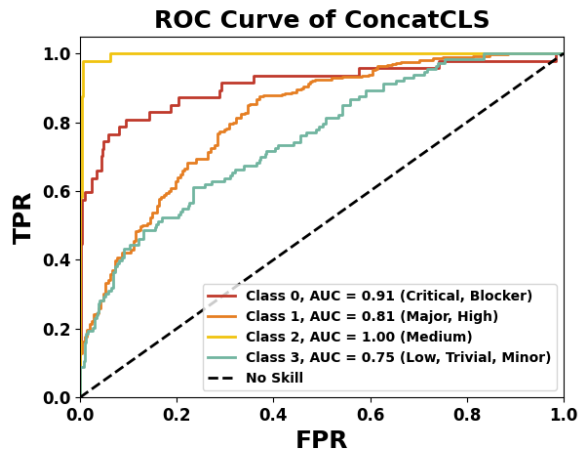
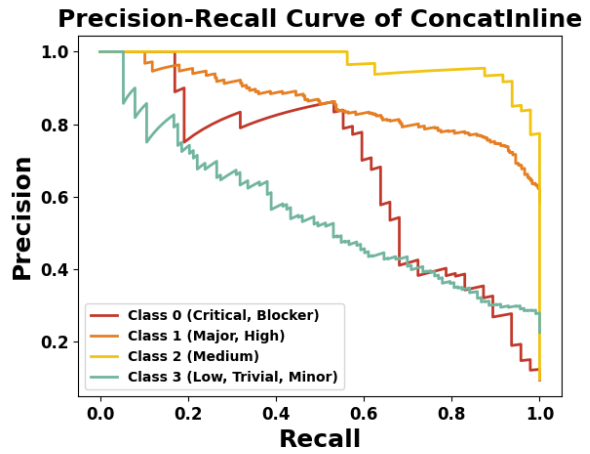
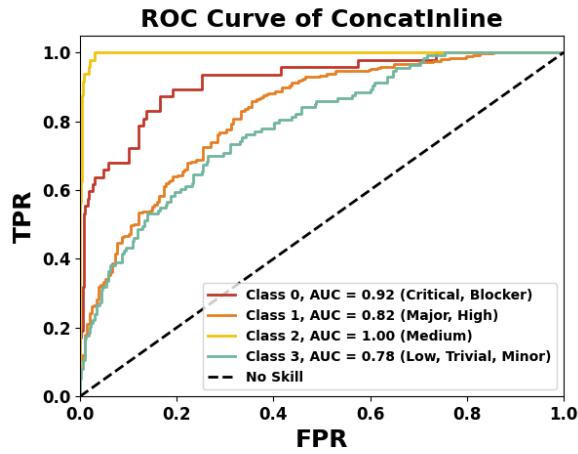


Figure 4.20: ROC curve and Precision-Recall curve of ConcatInline and ConcatCLS models.

indicator of bug severity which indicates there is no relationship between the size of a buggy method and its severity. Our findings from this RQ suggest that developers may still avoid long and complex methods to reduce the chance of bugs, but they may expect severe bugs even in the simple method with few line of codes.

Our RQ2 results show that studied static analysis tools (SpotBugs and Infer) can detect neither the buggy methods nor their severity. However, since these tools are used extensively in practice, many of the detected bugs may have been fixed soon during the development process by developers already. Our results imply that they are not practical in case of real complex problems, and there is a potential gap for the research community and the developers to study and provide more complex bug patterns or a combination of different approaches to detect real bugs. Furthermore, the reported severity values by SpotBugs show that it mislabels many of the severe bugs which reduces the usability of this tool in finding the bug severity, so this implies that the implemented approach by this tool, which is assigning the predefined severity value based on the bug pattern, is not practical and different conditions should be considered in determining the severity of a specific bug.

The results from RQ3 show that Security bugs have high severity in most cases while the Edge/Boundary bugs have low severity. We found that the bug type is a better indication of the bug severity in contrast to the bug complexity. Also, we found that SpotBugs tool does not have enough bug patterns to predict many of our studied severe bugs while it predicts lower severity values in most of the cases. However, we found that source code metrics and static analysis tools leverage different characteristics of the bugs, so leveraging both of them is more helpful.

Our results from RQ4 indicate that using only code metrics to train machine learning models do not lead to good results and in most cases, the trained classifiers are working like random classifier. The Random Forest model has the best result by having the F1-Weighted=0.54, AUC=0.65, and MCC=0.21. However, using the pre-trained CodeBERT model improves the result significantly regarding all evaluation metrics. This improvement shows that pre-trained models are powerful enough even in downstream tasks that are not part of their pre-training objective such as a severity prediction task. However, we find that adding numerical features by adding the numerical features in the textual format or concatenating the code metrics vector with the CLS token representation improves the CodeBERT model to a small extent. This relatively small improvement may be affected by our simple added layers compared to the very complicated layers in the CodeBERT model. Therefore, integrating more complex layers may improve the results even more. Our findings from this RQ suggest that having more training samples and also more code metrics would enable us to provide a more complex architecture that can improve the results.

Our findings can be used by the research community and the developers, so that they leverage other code

metrics to see their capability in finding bug severity, or perhaps other combinations of several code metrics may be found more useful. Also, our findings can be used for improving the quality of static analysis tools, especially in the case of the studied missed bugs. Developers should insist on writing simple methods to prevent bugs, but they may consider different conditions (the potential consequences on the overall system such as leaking sensitive information or crashing the whole system) during the development phase in order to reduce the risk of having high-severity bugs. Also, our results on the CodeBERT model can help others as a baseline for more complex models, or perhaps other information such as bug reports or code comments can be integrated into this model to find potential improvement.

4.4 Threats to Validity

There are some threats that may impact our results, so in this section, we provide possible threats and our actions to mitigate these threats.

4.4.1 External Validity

Our selected projects may not be consistent with the closed-source projects or may not cover various software domains or other programming languages. We did our best to select two large and popular datasets containing real-world bugs of a diverse set of open-source projects which have been used in many studies [77, 110, 98, 108]. The next threat would be the selection of static analysis tools, and for mitigating this problem we used the most popular tools which are used in practice and state-of-the-art research. We exclude Google Error Prone from the list of our tools since we found that it is not possible to handle older Java versions (exist in our dataset) with the latest versions of this tool. Also, the selected code metrics may not reveal the potential of all available code metrics in finding bug severity and this may impact our result generalizability, but to mitigate this issue we tried to leverage as many as available method-level code metrics that are used in previous research for defect-related problems (e.g, defect prediction). Also, our selected baseline models may not reveal the best performance of code metrics in predicting the bug severity labels. Since we found no previous study using code metrics for predicting bug severity to use their approach as a baseline and for mitigating this issue we selected most of the popular models which work well in similar tasks such as bug prediction. Our provided approach and the obtained results may not be robust against the cases where the provided patch in each instance contains bugs itself. For mitigating this issue we relied on our selected datasets where each buggy version has a corresponding fixed version that only fixes those issues, but in case of a possible new bug in the patches, a new method such as removing those instances in the pre-processing step should be performed.

4.4.2 Internal Validity

Another threat would be our strategy in selecting the buggy methods since we selected all methods that are changed/removed during the bug fixing patch as a buggy method which is a common approach in the state-of-the-art approaches (referenced in 4.1). The reason behind this is that bug fixing usually requires changing multiple methods and specifying one single method to be the source of a bug requires other information such as a bug report description. For mitigating this threat we consider each of those methods as an independent sample, so code metrics and static analysis tools can consider them independently. The next threat is the method of identifying reported bugs by static analyzers' tools. In this study, we consider the tools' report at the method-level granularity without considering the exact reported lines by these tools. We did this because there is no information regarding the exact bug location (line) in our datasets, and we want to keep the same method-granularity for RQ1 and RQ2. In order to mitigate this threat whenever any of our tools report a bug inside the method, we count it as a correct detection to not underestimate the tools' performance. Another issue would be our decision not to perform any hyper-parameter tuning which is out of this study's scope. This decision may have prevented our selected models to show their best performance, but to make the comparison doable and fair enough, we avoid any hyper-parameter tuning steps for all of our models.

4.4.3 Construct Validity

When calculating the sum complexity for answering RQ3-1, we have provided the same weight to all the different code metrics. These can be inaccurate because some code metrics can be more important than others. Through our manual observation, however, we found that a more complex method based on the sum complexity has higher complexity values for all the code metrics than a less complex method. Another issue would be the process of integrating numerical features into the CodeBERT model which may affect our results, so for not changing the baseline CodeBERT model structure drastically, we did not add more many complex layers to make the comparison fair. However, it is possible to leverage numerical features and integrate them with the CodeBERT model in a more complex way which may improve the results.

4.4.4 Conclusion Validity

Despite the efforts made though, these results may not be generalized using other tools and datasets. Our results of static analysis tools may be affected by using projects that have already applied the studied static analyzer tools warnings during the software development life cycle. Since it is not easy to understand if it happened, our results should be considered as an assessment of those bugs that are not already fixed by tools. Our selection of code metrics may not be sufficient to expose the bug severity, but for mitigating this

issue we considered the 10 popular metrics which have established performance in finding bugs in our study.

Chapter 5

Conclusion and Future Work

Detecting and fixing bugs are among the main maintenance activities in the software life cycle. However, among many bugs that potentially exist in the code the high severity ones are in great interest for the developers since their consequences are the most. Unfortunately, most existing studies in defect detection take all bugs the same and ignore their severity. In this study, we studied 10 source code metrics and two static analysis tools, as common methods for predicting buggy code, to find their capability in estimating the bugs' severity levels. Our quantitative and qualitative studies on Defects4J and Bugs.jar datasets containing 3,358 buggy methods from 19 projects of Java open-source projects, showed that code metrics are good indicators of buggyness (Line of Code, Maintainable Index, Fan-out, and Effort are the best metrics), but neither code metrics nor static analysis tools are significant estimators of bugs severity. We found that there is no relationship between the code complexity and bug severity, and static analysis tools miss many bugs due to the lack of complex defined patterns.

Manual inspection of severe bugs reveals that the Security and Edge/Boundary bug types have high and low severity in most cases, respectively. Finally, we showed that code metrics and static analysis tools leveraged different characteristics of the code, so they can be complementary to predict the bug severity.

Baseline models trained on source code metrics solely do not show promising results and they act like a random classifier in many cases while the Decision Tree and Random Forest work better than the others. However, leveraging the CodeBERT pre-trained model by using only the source code input improves the baseline models significantly. Furthermore, integrating source code metrics into the CodeBERT model by ConcatInline and ConcatCLS models improves the CodeBERT model, and the results show that ConcatCLS has better efficacy than the ConcatInline model.

Potential future directions of this research are studying the power of dynamic analysis and testing in

estimating bug severity. In addition, one can use the findings of this study regarding static analysis tools limitations and try to enrich their rule sets to better identify severe bugs. Also, other available features such as code comments or bug reports can be used in addition to the code metrics in training the models to find their effectiveness in predicting bug severity. Finally, as another potential future work, integrating the inherent structure of code (e.g., data flow, control flow, etc.) into the CodeBERT model (or using the existing GraphCodeBERT) may enhance its efficacy.

Bibliography

- [1] Bug severity. URL <https://www.browserstack.com/guide/bug-severity-vs-priority>
- [2] Classification metrics. URL <https://machinelearningmastery.com/tour-of-evaluation-metrics-for-imbalanced-classification/>. [Online; accessed 20-April-2022]
- [3] Errorprone. URL <https://errorprone.info/>. [Online; accessed 20-April-2022]
- [4] Findbugs. URL <http://findbugs.sourceforge.net/>. [Online; accessed 20-April-2022]
- [5] Hugging face. URL <https://huggingface.co/>. [Online; accessed 20-April-2022]
- [6] Infer. URL <https://fbinfer.com/>. [Online; accessed 20-April-2022]
- [7] Macroaverage. URL https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html. [Online; accessed 20-April-2022]
- [8] Robustscaler. URL <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.RobustScaler.html>. [Online; accessed 20-April-2022]
- [9] Spotbugs. URL <https://spotbugs.github.io/>. [Online; accessed 20-April-2022]
- [10] Visual studio3. URL <https://docs.microsoft.com/en-us/visualstudio/code-quality/code-metrics-maintainability-index-range-and-meaning?view=vs-2022&viewFallbackFrom=vs-2022%3A>. [Online; accessed 20-April-2022]
- [11] Akila, V., Zayaraz, G., Govindasamy, V.: Effective bug triage—a framework. *Procedia Computer Science* **48**, 114–120 (2015)
- [12] Alenezi, M., Akour, M., Al Sghaier, H.: The impact of co-evolution of code production and test suites through software releases in open source software systems. *International Journal of Innovative Technology and Exploring Engineering (IJITEE)* **9**(1), 2737–2739 (2019)

- [13] AlOmar, E.A., Mkaouer, M.W., Ouni, A., Kessentini, M.: On the impact of refactoring on the relationship between quality attributes and design metrics. In: 2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), pp. 1–11. IEEE (2019)
- [14] Alon, U., Zilberstein, M., Levy, O., Yahav, E.: code2vec: Learning distributed representations of code. *Proceedings of the ACM on Programming Languages* **3**(POPL), 1–29 (2019)
- [15] Aniche, M.F., Oliva, G.A., Gerosa, M.A.: What do the asserts in a unit test tell us about code quality? a study on open source and industrial projects. In: 2013 17th European Conference on Software Maintenance and Reengineering, pp. 111–120. IEEE (2013)
- [16] Antinyan, V., Staron, M., Meding, W., Österström, P., Wikstrom, E., Wrangler, J., Henriksson, A., Hansson, J.: Identifying risky areas of software code in agile/lean software development: An industrial experience report. In: 2014 Software Evolution Week-IEEE Conference on Software Maintenance, Reengineering, and Reverse Engineering (CSMR-WCRE), pp. 154–163. IEEE (2014)
- [17] Antinyan, V., Staron, M., Sandberg, A.: Evaluating code complexity triggers, use of complexity measures and the influence of code complexity on maintenance time. *Empirical Software Engineering* **22**(6), 3057–3087 (2017)
- [18] Ayewah, N., Pugh, W., Morgenthaler, J.D., Penix, J., Zhou, Y.: Using findbugs on production software. In: Companion to the 22nd ACM SIGPLAN conference on Object-oriented programming systems and applications companion, pp. 805–806 (2007)
- [19] Bacchelli, A., Bird, C.: Expectations, outcomes, and challenges of modern code review. In: 2013 35th International Conference on Software Engineering (ICSE), pp. 712–721. IEEE (2013)
- [20] Bacchelli, A., D’Ambros, M., Lanza, M.: Are popular classes more defect prone? In: International Conference on Fundamental Approaches to Software Engineering, pp. 59–73. Springer (2010)
- [21] Bavota, G., De Carluccio, B., De Lucia, A., Di Penta, M., Oliveto, R., Strollo, O.: When does a refactoring induce bugs? an empirical study. In: 2012 IEEE 12th International Working Conference on Source Code Analysis and Manipulation, pp. 104–113. IEEE (2012)
- [22] Bengio, Y., Courville, A., Vincent, P.: Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence* **35**(8), 1798–1828 (2013)
- [23] Bennett, K.H., Rajlich, V.T.: Software maintenance and evolution: a roadmap. In: Proceedings of the Conference on the Future of Software Engineering, pp. 73–87 (2000)

- [24] Bhat, T., Nagappan, N.: Evaluating the efficacy of test-driven development: industrial case studies. In: Proceedings of the 2006 ACM/IEEE international symposium on Empirical software engineering, pp. 356–363 (2006)
- [25] Börstler, J., Paech, B.: The role of method chains and comments in software readability and comprehension—an experiment. *IEEE Transactions on Software Engineering* **42**(9), 886–898 (2016)
- [26] Buitinck, L., Louppe, G., Blondel, M., Pedregosa, F., Mueller, A., Grisel, O., Niculae, V., Prettenhofer, P., Gramfort, A., Grobler, J., et al.: Api design for machine learning software: experiences from the scikit-learn project. *arXiv preprint arXiv:1309.0238* (2013)
- [27] Buse, R.P., Weimer, W.R.: Learning a metric for code readability. *IEEE Transactions on software engineering* **36**(4), 546–558 (2009)
- [28] Catolino, G., Palomba, F., Zaidman, A., Ferrucci, F.: Not all bugs are the same: Understanding, characterizing, and classifying bug types. *Journal of Systems and Software* **152**, 165–181 (2019)
- [29] Chaturvedi, K.K., Singh, V.: Determining bug severity using machine learning techniques. In: 2012 CSI sixth international conference on software engineering (CONSEG), pp. 1–6. IEEE (2012)
- [30] Chillarege, R., Bhandari, I.S., Chaar, J.K., Halliday, M.J., Moebus, D.S., Ray, B.K., Wong, M.Y.: Orthogonal defect classification-a concept for in-process measurements. *IEEE Transactions on software Engineering* **18**(11), 943–956 (1992)
- [31] Chowdhury, S., Uddin, G., Holmes, R.: An empirical study on maintainable method size in java. In: 19th International Conference on Mining Software Repositories (2022)
- [32] Cliff, N.: Dominance statistics: Ordinal analyses to answer ordinal questions. *Psychological bulletin* **114**(3), 494 (1993)
- [33] Curtis, B., Sheppard, S.B., Milliman, P., Borst, M., Love, T.: Measuring the psychological complexity of software maintenance tasks with the halstead and mccabe metrics. *IEEE Transactions on software engineering* (2), 96–104 (1979)
- [34] Denzin, N.K., Lincoln, Y.S.: The Sage handbook of qualitative research. sage (2011)
- [35] Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805* (2018)
- [36] Dura, A., Reichenbach, C., Söderberg, E.: Javadl: automatically incrementalizing java bug pattern detection. *Proceedings of the ACM on Programming Languages* **5**(OOPSLA), 1–31 (2021)

- [37] El Emam, K., Benlarbi, S., Goel, N., Rai, S.N.: The confounding effect of class size on the validity of object-oriented metrics. *IEEE Transactions on Software Engineering* **27**(7), 630–650 (2001)
- [38] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al.: Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020)
- [39] Ferenc, R., Bán, D., Grósz, T., Gyimóthy, T.: Deep learning in static, metric-based bug prediction. *Array* **6**, 100021 (2020)
- [40] Ferenc, R., Gyimesi, P., Gyimesi, G., Tóth, Z., Gyimóthy, T.: An automatically created novel bug dataset and its validation in bug prediction. *The Journal of systems and software* **169**, 110691 (2020)
- [41] Giger, E., D’Ambros, M., Pinzger, M., Gall, H.C.: Method-level bug prediction. In: *Proceedings of the 2012 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, pp. 171–180. IEEE (2012)
- [42] Giger, E., Pinzger, M., Gall, H.C.: Comparing fine-grained source code changes and code churn for bug prediction. In: *Proceedings of the 8th Working Conference on Mining Software Repositories*, pp. 83–92 (2011)
- [43] Gil, Y., Lalouche, G.: On the correlation between size and metric validity. *Empirical Software Engineering* **22**(5), 2585–2611 (2017)
- [44] Grund, F., Chowdhury, S., Bradley, N.C., Hall, B., Holmes, R.: Codeshovel: Constructing method-level source code histories. In: *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pp. 1510–1522. IEEE (2021)
- [45] Habib, A., Pradel, M.: How many of all bugs do we find? a study of static bug detectors. In: *2018 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, pp. 317–328. IEEE (2018)
- [46] Habib, A., Pradel, M.: Neural bug finding: A study of opportunities and challenges. *arXiv preprint arXiv:1906.00307* (2019)
- [47] Hall, T., Beecham, S., Bowes, D., Gray, D., Counsell, S.: A systematic literature review on fault prediction performance in software engineering. *IEEE Transactions on Software Engineering* **38**(6), 1276–1304 (2011)

- [48] Halstead, M.H.: Elements of Software Science (Operating and programming systems series). Elsevier Science Inc. (1977)
- [49] Harer, J.A., Kim, L.Y., Russell, R.L., Ozdemir, O., Kosta, L.R., Rangamani, A., Hamilton, L.H., Centeno, G.I., Key, J.R., Ellingwood, P.M., et al.: Automated software vulnerability detection with machine learning. arXiv preprint arXiv:1803.04497 (2018)
- [50] Hata, H., Mizuno, O., Kikuno, T.: Bug prediction based on fine-grained module histories. In: 2012 34th international conference on software engineering (ICSE), pp. 200–210. IEEE (2012)
- [51] Hemmati, H.: How effective are code coverage criteria? In: 2015 IEEE International Conference on Software Quality, Reliability and Security, pp. 151–156. IEEE (2015)
- [52] Herzig, K., Zeller, A.: The impact of tangled code changes. In: 2013 10th Working Conference on Mining Software Repositories, pp. 121–130 (2013)
- [53] Hess, M.R., Kromrey, J.D.: Robust confidence intervals for effect sizes: A comparative study of cohen’sd and cliff’s delta under non-normality and heterogeneous variances. In: annual meeting of the American Educational Research Association, vol. 1. Citeseer (2004)
- [54] Hindle, A., Godfrey, M.W., Holt, R.C.: Reading beside the lines: Indentation as a proxy for complexity metric. In: 2008 16th IEEE International Conference on Program Comprehension, pp. 133–142. IEEE (2008)
- [55] Hosseini, S., Turhan, B., Gunarathna, D.: A systematic literature review and meta-analysis on cross project defect prediction. IEEE Transactions on Software Engineering **45**(2), 111–147 (2017)
- [56] Huang, X., Zhang, H., Zhou, X., Babar, M.A., Yang, S.: Synthesizing qualitative research in software engineering: A critical review. In: Proceedings of the 40th international conference on software engineering, pp. 1207–1218 (2018)
- [57] Husain, H., Wu, H.H., Gazit, T., Allamanis, M., Brockschmidt, M.: Codesearchnet challenge: Evaluating the state of semantic code search. arXiv preprint arXiv:1909.09436 (2019)
- [58] Jureczko, M., Spinellis, D.: Using object-oriented design metrics to predict software defects. Models and Methods of System Dependability. Oficyna Wydawnicza Politechniki Wrocławskiej pp. 69–81 (2010)
- [59] Just, R., Jalali, D., Ernst, M.D.: Defects4j: A database of existing faults to enable controlled testing studies for java programs. In: Proceedings of the 2014 International Symposium on Software Testing and Analysis, pp. 437–440 (2014)

- [60] Kafura, D., Reddy, G.R.: The use of software complexity metrics in software maintenance. *IEEE Transactions on Software Engineering* (3), 335–343 (1987)
- [61] Kanwal, J., Maqbool, O.: Bug prioritization to facilitate bug report triage. *Journal of Computer Science and Technology* **27**(2), 397–412 (2012)
- [62] Kasto, N., Whalley, J.: Measuring the difficulty of code comprehension tasks using software metrics. In: *Proceedings of the Fifteenth Australasian Computing Education Conference-Volume 136*, pp. 59–65 (2013)
- [63] Khatiwada, S., Tushev, M., Mahmoud, A.: Just enough semantics: An information theoretic approach for ir-based software bug localization. *Information and Software Technology* **93**, 45–57 (2018)
- [64] Khomh, F., Penta, M.D., Guéhéneuc, Y.G., Antoniol, G.: An exploratory study of the impact of antipatterns on class change-and fault-proneness. *Empirical Software Engineering* **17**(3), 243–275 (2012)
- [65] Kondo, M., German, D.M., Mizuno, O., Choi, E.H.: The impact of context metrics on just-in-time defect prediction. *Empirical Software Engineering* **25**(1), 890–939 (2020)
- [66] Kononenko, O., Baysal, O., Godfrey, M.W.: Code review quality: How developers see it. In: *Proceedings of the 38th international conference on software engineering*, pp. 1028–1038 (2016)
- [67] Koru, A., Liu, H.: Building effective defect-prediction models in practice. *IEEE Software* **22**(6), 23–29 (2005)
- [68] Landman, D., Serebrenik, A., Vinju, J.: Empirical analysis of the relationship between cc and sloc in a large corpus of java methods. In: *2014 IEEE International Conference on Software Maintenance and Evolution*, pp. 221–230. IEEE (2014)
- [69] Le, Q., Mikolov, T.: Distributed representations of sentences and documents. In: *International conference on machine learning*, pp. 1188–1196. PMLR (2014)
- [70] Le Goues, C., Nguyen, T., Forrest, S., Weimer, W.: Genprog: A generic method for automatic software repair. *Ieee transactions on software engineering* **38**(1), 54–72 (2011)
- [71] Li, N., Shepperd, M., Guo, Y.: A systematic review of unsupervised learning techniques for software defect prediction. *Information and Software Technology* **122**, 106287 (2020)
- [72] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692* (2019)

- [73] Long, F., Rinard, M.: Automatic patch generation by learning correct code. In: Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, pp. 298–312 (2016)
- [74] Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., et al.: Codexglue: A machine learning benchmark dataset for code understanding and generation. arXiv preprint arXiv:2102.04664 (2021)
- [75] Mäntylä, M.V., Lassenius, C.: What types of defects are really discovered in code reviews? IEEE Transactions on Software Engineering **35**(3), 430–448 (2008)
- [76] Martin, R.C.: Professionalism and test-driven development. Ieee Software **24**(3), 32–36 (2007)
- [77] Martinez, M., Durieux, T., Sommerard, R., Xuan, J., Monperrus, M.: Automatic repair of real bugs in java: A large-scale experiment on the defects4j dataset. Empirical Software Engineering **22**(4), 1936–1964 (2017)
- [78] Mashhadi, E., Chowdhury, S., Modaberi, S., Hemmati, H., Uddin, G.: An empirical study on bug severity estimation using source code metrics and static analysis. arXiv preprint arXiv:2206.12927 (2022)
- [79] Mashhadi, E., Hemmati, H.: Applying codebert for automated program repair of java simple bugs. In: 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR), pp. 505–509. IEEE (2021)
- [80] Matter, D., Kuhn, A., Nierstrasz, O.: Assigning bug reports using a vocabulary-based expertise model of developers. In: 2009 6th IEEE International Working Conference on Mining Software Repositories, pp. 131–140 (2009)
- [81] Matthews, B.W.: Comparison of the predicted and observed secondary structure of t4 phage lysozyme. Biochimica et Biophysica Acta (BBA)-Protein Structure **405**(2), 442–451 (1975)
- [82] McCabe, T.J.: A complexity measure. IEEE Transactions on software Engineering (4), 308–320 (1976)
- [83] McClure, C.L.: A model for program complexity analysis. In: Proceedings of the 3rd international conference on Software engineering, pp. 149–157 (1978)
- [84] Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781 (2013)

- [85] Miles, M.B., Huberman, A.M.: Qualitative data analysis: An expanded sourcebook. sage (1994)
- [86] Mo, R., Cai, Y., Kazman, R., Xiao, L., Feng, Q.: Decoupling level: a new metric for architectural maintenance complexity. In: 2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE), pp. 499–510. IEEE (2016)
- [87] Mo, R., Wei, S., Feng, Q., Li, Z.: An exploratory study of bug prediction at the method level. *Information and software technology* **144**, 106794 (2022)
- [88] Nawrocki, J., Wojciechowski, A.: Experimental evaluation of pair programming. *European Software Control and Metrics (Escom)* pp. 99–101 (2001)
- [89] Neysiani, B.S., Babamir, S.M., Aritsugi, M.: Efficient feature extraction model for validation performance improvement of duplicate bug report detection in software bug triage systems. *Information and Software Technology* **126**, 106344 (2020)
- [90] Nguyen, T.D., Nguyen, A.T., Nguyen, T.N.: Mapping api elements for code migration with vector representations. In: 2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C), pp. 756–758. IEEE (2016)
- [91] Okutan, A., Yıldız, O.T.: Software defect prediction using bayesian networks. *Empirical software engineering : an international journal* **19**(1), 154–181 (2014)
- [92] Oman, P., Hagemeister, J.: Metrics for assessing a software system’s maintainability. In: *Proceedings Conference on Software Maintenance 1992*, pp. 337–338. IEEE Computer Society (1992)
- [93] Pai, G.J., Dugan, J.B.: Empirical analysis of software fault content and fault proneness using bayesian methods. *IEEE Transactions on software Engineering* **33**(10), 675–686 (2007)
- [94] Palomba, F., Bavota, G., Penta, M.D., Fasano, F., Oliveto, R., Lucia, A.D.: On the diffuseness and the impact on maintainability of code smells: a large scale empirical investigation. *Empirical Software Engineering* **23**(3), 1188–1221 (2018)
- [95] Pan, C., Lu, M., Xu, B.: An empirical study on software defect prediction using codebert model. *Applied Sciences* **11**(11), 4793 (2021)
- [96] Pantiuchina, J., Lanza, M., Bavota, G.: Improving code: The (mis) perception of quality metrics. In: *2018 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pp. 80–91. IEEE (2018)

- [97] Pascarella, L., Palomba, F., Bacchelli, A.: On the performance of method-level bug prediction: A negative result. *Journal of Systems and Software* **161**, 110493 (2020)
- [98] Pearson, S., Campos, J., Just, R., Fraser, G., Abreu, R., Ernst, M.D., Pang, D., Keller, B.: Evaluating and improving fault localization. In: 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE), pp. 609–620. IEEE (2017)
- [99] Pecorelli, F., Palomba, F., Di Nucci, D., De Lucia, A.: Comparing heuristic and machine learning approaches for metric-based code smell detection. In: 2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC), pp. 93–104. IEEE (2019)
- [100] Polo, M., Piattini, M., Ruiz, F.: Using code metrics to predict maintenance of legacy programs: A case study. In: Proceedings IEEE International Conference on Software Maintenance. ICSM 2001, pp. 202–208. IEEE (2001)
- [101] Posnett, D., Hindle, A., Devanbu, P.: A simpler model of software readability. In: Proceedings of the 8th working conference on mining software repositories, pp. 73–82 (2011)
- [102] Rafique, Y., Mišić, V.B.: The effects of test-driven development on external quality and productivity: A meta-analysis. *IEEE Transactions on Software Engineering* **39**(6), 835–856 (2012)
- [103] Rahman, A., Williams, L.: Source code properties of defective infrastructure as code scripts. *Information and Software Technology* **112**, 148–163 (2019)
- [104] Ralph, P., Tempero, E.: Construct validity in software engineering research and software metrics. In: Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018, pp. 13–23 (2018)
- [105] Ramay, W.Y., Umer, Q., Yin, X.C., Zhu, C., Illahi, I.: Deep neural network-based severity prediction of bug reports. *IEEE Access* **7**, 46846–46857 (2019)
- [106] Saha, R.K., Khurshid, S., Perry, D.E.: An empirical study of long lived bugs. In: 2014 Software Evolution Week-IEEE Conference on Software Maintenance, Reengineering, and Reverse Engineering (CSMR-WCRE), pp. 144–153. IEEE (2014)
- [107] Saha, R.K., Lyu, Y., Lam, W., Yoshida, H., Prasad, M.R.: Bugs. jar: a large-scale, diverse dataset of real-world java bugs. In: Proceedings of the 15th international conference on mining software repositories, pp. 10–13 (2018)

- [108] Saha, R.K., Lyu, Y., Yoshida, H., Prasad, M.R.: Elixir: Effective object-oriented program repair. In: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 648–659. IEEE (2017)
- [109] Servant, F., Jones, J.A.: Fuzzy fine-grained code-history analysis. In: Proceedings of the International Conference on Software Engineering (ICSE), pp. 746–757 (2017)
- [110] Shamshiri, S., Just, R., Rojas, J.M., Fraser, G., McMinn, P., Arcuri, A.: Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges (t). In: 2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 201–211. IEEE (2015)
- [111] Shaphiro, S., Wilk, M.: An analysis of variance test for normality. *Biometrika* **52**(3), 591–611 (1965)
- [112] Shihab, E., Hassan, A.E., Adams, B., Jiang, Z.M.: An industrial study on the risk of software changes. In: Proceedings of the ACM SIGSOFT 20th International Symposium on the Foundations of Software Engineering, pp. 1–11 (2012)
- [113] Shin, Y., Meneely, A., Williams, L., Osborne, J.A.: Evaluating complexity, code churn, and developer activity metrics as indicators of software vulnerabilities. *IEEE transactions on software engineering* **37**(6), 772–787 (2010)
- [114] Shivaji, S., Whitehead, E.J., Akella, R., Kim, S.: Reducing features to improve bug prediction. In: 2009 IEEE/ACM International Conference on Automated Software Engineering, pp. 600–604. IEEE (2009)
- [115] Sjøberg, D.I.K., Yamashita, A., Anda, B.C.D., Mockus, A., Dybå, T.: Quantifying the effect of code smells on maintenance effort. *IEEE Transactions on Software Engineering* **39**(8), 1144–1156 (2013)
- [116] Spadini, D., Palomba, F., Zaidman, A., Bruntink, M., Bacchelli, A.: On the relation of test smells to software code quality. In: 2018 IEEE international conference on software maintenance and evolution (ICSME), pp. 1–12. IEEE (2018)
- [117] Steidl, D., Hummel, B., Juergens, E.: Incremental origin analysis of source code files. In: Proceedings Working Conference on Mining Software Repositories (MSR), pp. 42 – 51 (2014)
- [118] Subramanyam, R., Krishnan, M.S.: Empirical analysis of ck metrics for object-oriented design complexity: Implications for software defects. *IEEE Transactions on software engineering* **29**(4), 297–310 (2003)

- [119] Sun, W., Marakas, G., Aguirre-Urreta, M.: The effectiveness of pair programming: Software professionals' perceptions. *IEEE Software* **33**(4), 72–79 (2015)
- [120] Sun, X., Zhou, W., Li, B., Ni, Z., Lu, J.: Bug localization for version issues with defect patterns. *IEEE Access* **7**, 18811–18820 (2019)
- [121] Tan, Y., Xu, S., Wang, Z., Zhang, T., Xu, Z., Luo, X.: Bug severity prediction using question-and-answer pairs from stack overflow. *Journal of Systems and Software* **165**, 110567 (2020)
- [122] Tian, Y., Lo, D., Sun, C.: Information retrieval based nearest neighbor classification for fine-grained bug severity prediction. In: 2012 19th Working Conference on Reverse Engineering, pp. 215–224. IEEE (2012)
- [123] Tomassi, D.A.: Bugs in the wild: examining the effectiveness of static analyzers at finding real-world bugs. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 980–982 (2018)
- [124] Tosun, A., Bener, A., Turhan, B., Menzies, T.: Practical considerations in deploying statistical methods for defect prediction: A case study within the turkish telecommunications industry. *Information and Software Technology* **52**(11), 1242–1257 (2010)
- [125] Tufano, M., Palomba, F., Bavota, G., Oliveto, R., Di Penta, M., De Lucia, A., Shybyanyk, D.: When and why your code starts to smell bad. In: 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering, vol. 1, pp. 403–414. IEEE (2015)
- [126] Uddin, J., Ghazali, R., Deris, M.M., Naseem, R., Shah, H.: A survey on bug prioritization. *Artificial Intelligence Review* **47**(2), 145–180 (2017)
- [127] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Advances in neural information processing systems* **30** (2017)
- [128] Vucevic, D., Yaddow, W.: Testing the data warehouse practicum: Assuring data content, data structures and quality. Trafford Publishing (2012)
- [129] Wahono, R.S.: A systematic literature review of software defect prediction. *Journal of software engineering* **1**(1), 1–16 (2015)
- [130] Wang, D., Lin, M., Zhang, H., Hu, H.: Detect related bugs from source code using bug information. In: 2010 IEEE 34th Annual Computer Software and Applications Conference, pp. 228–237. IEEE (2010)

- [131] Wang, Q., Xia, X., Lo, D., Li, S.: Why is my code change abandoned? *Information and Software Technology* **110**, 108 – 120 (2019)
- [132] Watson, A.H., Wallace, D.R., McCabe, T.J.: Structured testing: A testing methodology using the cyclomatic complexity metric, vol. 500. US Department of Commerce, Technology Administration, National Institute of ... (1996)
- [133] Wattanakriengkrai, S., Thongtanunam, P., Tantithamthavorn, C., Hata, H., Matsumoto, K.: Predicting defective lines using a model-agnostic technique. *arXiv preprint arXiv:2009.03612* (2020)
- [134] Williams, L., Kessler, R.R., Cunningham, W., Jeffries, R.: Strengthening the case for pair programming. *IEEE software* **17**(4), 19–25 (2000)
- [135] Wong, W.E., Gao, R., Li, Y., Abreu, R., Wotawa, F.: A survey on software fault localization. *IEEE Transactions on Software Engineering* **42**(8), 707–740 (2016)
- [136] Yang, G., Zhang, T., Lee, B.: Towards semi-automatic bug triage and severity prediction based on topic model and multi-feature of bug reports. In: *2014 IEEE 38th Annual Computer Software and Applications Conference*, pp. 97–106. IEEE (2014)
- [137] Zaw, K.K., Hnin, H.W., Kyaw, K.Y., Funabiki, N.: Software quality metrics calculations for java programming learning assistant system. In: *2020 IEEE Conference on Computer Applications (ICCA)*, pp. 1–6. IEEE (2020)
- [138] Zhang, F., Yu, X., Keung, J., Li, F., Xie, Z., Yang, Z., Ma, C., Zhang, Z.: Improving stack overflow question title generation with copying enhanced codebert model and bi-modal information. *Information and Software Technology* **148**, 106922 (2022)
- [139] Zhang, T., Chen, J., Yang, G., Lee, B., Luo, X.: Towards more accurate severity prediction and fixer recommendation of software bugs. *Journal of Systems and Software* **117**, 166–184 (2016)
- [140] Zhou, Y., Xu, B., Leung, H.: On the ability of complexity metrics to predict fault-prone classes in object-oriented systems. *Journal of Systems and Software* **83**(4), 660–674 (2010)
- [141] Zimmermann, T., Premraj, R., Zeller, A.: Predicting defects for eclipse. In: *Third International Workshop on Predictor Models in Software Engineering (PROMISE’07: ICSE Workshops 2007)*, pp. 9–9. IEEE (2007)